

Adding Utilities

Created by Clarke, Jolyon M, last updated by Nataraj, Vishwas on Nov 19, 2025 • 71 minute read

SQLPathFinder allows you to add certain utilities to your query. These utilities includes running a Python script or a JMP/JSL script, e-mailing a results file to a distribution list, or publishing a file to a Sharepoint site.

This help topic has several sections.

1. [Adding a Utility to Your Query](#)
2. [Copy Utility - Adding or Moving a CSV File](#)
3. [Web-Copy Utility](#)
4. [Excel Utility - Formatting Output or Plotting Charts](#)
5. [Basic JMP Utility - Formatting Output or Plotting Charts](#)
6. [Emailing a File to a Distribution List](#)
7. [Publishing to SharePoint](#)
8. [Program-Block Utilities \(e.g., If-Then-Else, Run-Loop, Site-Loop, For-Loop, Start-Macro, HPC\).](#) Program-block utilities are meant utilities that have both a beginning point and an endpoint, and which affect all steps between those two points.
 - a. [If-Then-Else Utility](#)
 - b. [Application of the Count Rows in File and the If-Then-Else Utility to Conditionally Execute Steps](#)
 - c. [Application of the If-Then-Else and Value-In-File Utilities to Compare a User Name in a Configuration File to the "UserName" Environment Variable](#)
 - d. [Start-Macro and Macro variable substitution](#)
 - e. [Using the For-Loop and the Smart-Append Utilities to run an incremental job](#)
 - f. [Notes on the Site-Loop Utility](#)
 - g. [Notes on the Run-Loop Utility Used in Conjunction with Macros](#)
 - h. [Option for Program-Block Utilities - *Run Child Utils Even if Parent Utility is Disabled*](#)
 - i. [Begin-HPC and End-HPC Utilities for High Performance Computing](#)
 - j. [Expanding and Collapsing Utilities](#)
9. [Deleting, Moving, or Running a Block of Utilities](#)
10. [Determining the Age of a File as a precursor to some action](#)
11. [Smart Append of Data to a File \(Supports files with different columns/column orders, and with need for data updates and deletes\)](#)
12. [Other File Utilities - Simple Data Append, Deleting a File, Renaming a File, or Moving a File](#)
13. [Creating a Running "N" Day Extract File - Append New Data Since the Last Run, Delete Old Data, Update Changed Data](#)
14. [Use a Utility to Prompt for a collective set of inputs. This is for advanced users. Simple prompting for input can be done by checking the prompt option on the column filter form.](#)
15. [Write-File Utility](#)
16. [Date-of-File Utility](#)
17. [Python Utility](#)
18. [R Utility](#)
19. [Pre/Post Query Utility](#)
20. [Get-TCA Utility](#)
21. [TDX-TO-CSV Utility](#)
22. [Mongo-Import Utility](#)
23. [File-Compare Utility \(for GUI version 9.6.4.9 or higher\)](#)
24. [IDEAL-Table-Config Utility](#)
25. [Write-File and Write-File-Immediate Utilities](#)
26. [Screen-and-Visualize-Features](#)

Adding a Utility to Your Query

[\[Return to Top\]](#)

To add a utility to your query, from the Query design form choose menu option **Query -> Add Table or Utility**, and select **Data Area Utilities** on the **Add View or Utility** window. You will be shown a list of utilities as illustrated on Figure 1 below in the Copy Utility ... section. To add a utility, e.g., the *Copy Utility* as illustrated in Figure 1, double-click the utility name. The utility will then be added to the end of your query. Close the Add a View or Utility window to interact with the utility. Use drag-and-drop to position the utility in the query.

Copy Utility - Adding or Moving a CSV File

[\[Return to Top\]](#)

SQLPathFinder allows you to copy files from one area to another prior to or after a query runs. This can be useful to copy a tab or comma delimited file to the SQLPathFinder working directory prior to processing the file, or to copy a result file to a file share after a query completes.

From the Query design form, choose menu option **Query -> Add Table or Utility**, select **Data Area Utilities** on the **Add a Table** window, and double-click the **Copy** item to add a *Copy Utility* to your query. See Figure 1 below. You can position the utility using drag-and-drop.

Double-click the *Copy Utility* in the Query Design Tree control to configure it. The *Utilities - Copy* form is invoked which requires a source and destination file. See Figure 2 below. If no destination path is specified, the file will be copied to your SQLPathFinder default working directory for both an interactive or a ScriptHost job. Similarly, if no source path is specified, the file will be copied from your default SQLPathFinder work folder. The Copy Utility also allows you to check option **Continue Job if Copy Fails** if you wish the SQLPathFinder job to continue even if the copy action fails. Click **OK** to commit your changes and return to the Query Design form.

Figure 1 - Adding a Copy File Utility to Your Query

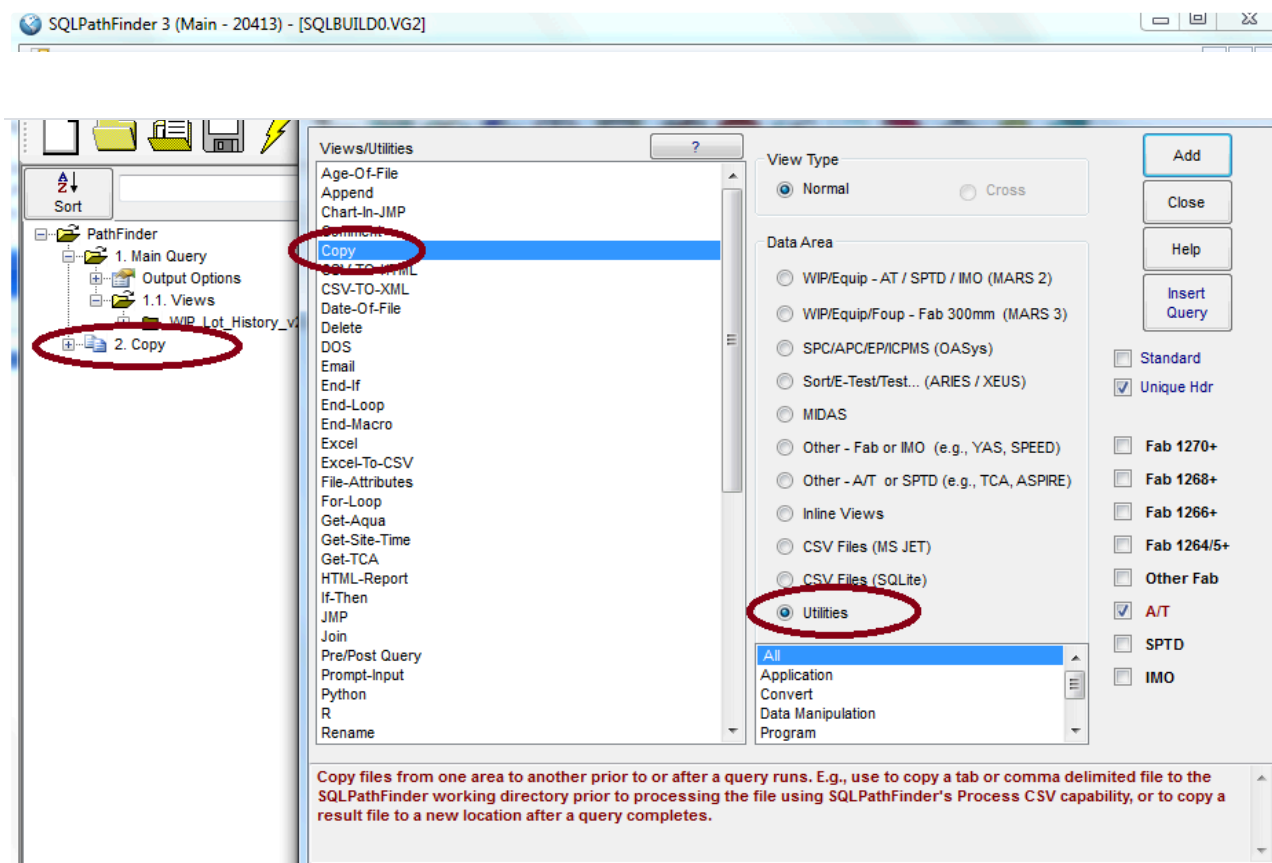
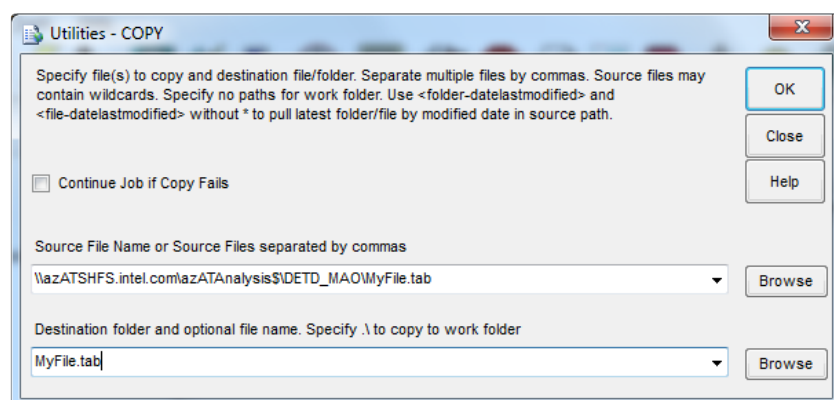


Figure 2 - The Copy Utility Form


[\[Return to Top\]](#)

Web-Copy Utility - Retrieving a File from a Web Site

[\[Return to Top\]](#)

SQLPathFinder allows you to copy binary or text files from a web site. This can be especially important on ScriptHost where you are restricted on the file shares you can access. However you will be able to read and write from any web site on the Intel Intranet. Placing files on a web site also allows you to secure that file.

From the **Add View or Utility** form, select **Data Area Utilities**, and double-click the **Web-Copy** item to add a **Web-Copy Utility** to your query. See Figure 3a below. Position the utility using drag-and-drop.

Double-click the **Web-Copy Utility** in the Query Design form to configure it. The **Utilities - Web-Copy** form (see Figure 3b) requires a file web source you wish to copy, a destination path and whether an ASCII or Binary copy should be done. If a destination file without a path is specified, the file will be copied to your SQLPathFinder work folder. Click **OK** to commit your changes.

Figure 3a - Adding a Web-Copy Utility to Your Query

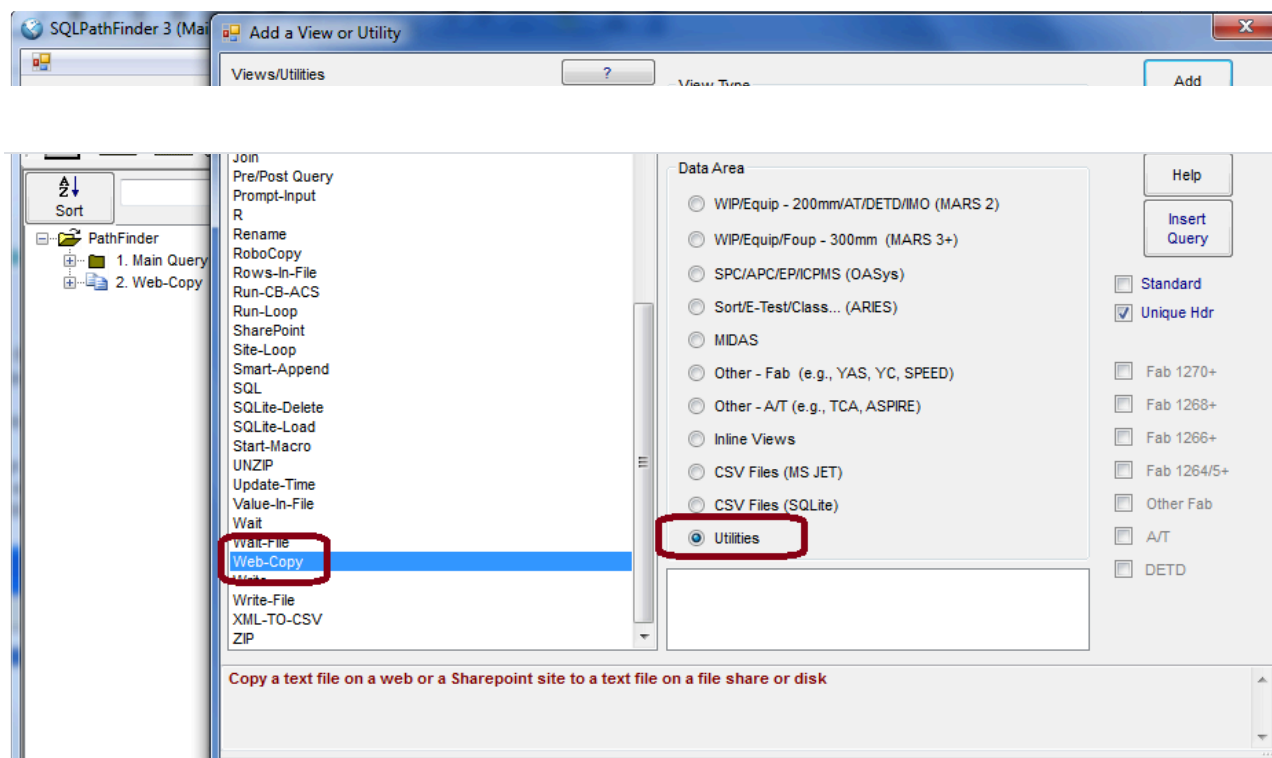
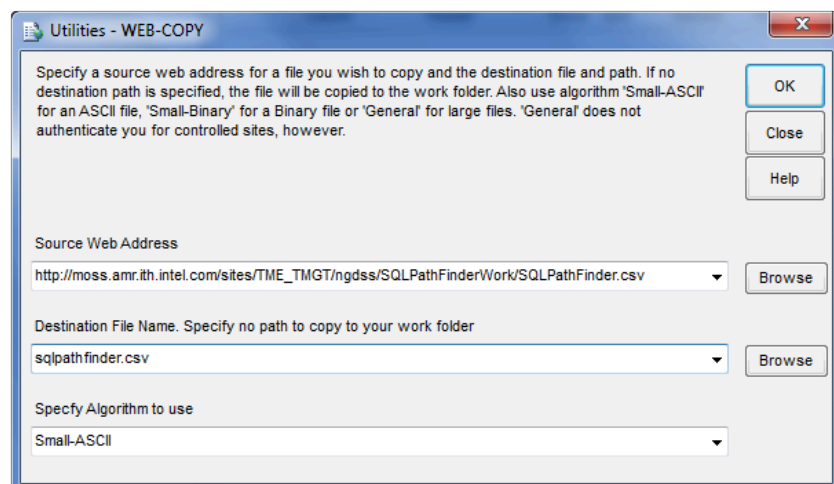


Figure 3b - The Web-Copy Utility Form


[\[Return to Top\]](#)

Excel Utility - Formatting Output or Plotting Charts

[\[Return to Top\]](#)

Important Note on Using the Excel Utility

Microsoft notes that, "all current versions of Microsoft Office were designed, tested, and configured to run as end-user products on a client workstation." "Microsoft also does not recommend or support server-side Automation of Office." Microsoft further notes that, "a number of complications can occur if Automation takes place from server-side code such as a Windows NT service." This includes Task Manager. This is discussed in this article: <https://support.microsoft.com/en-us/help/257757/considerations-for-server-side-automation-of-office>

Furthermore, the ScriptHost team will only support running Excel macros in unattended mode as best effort. Whenever there is an Operating System upgrade, we risk losing this capability of running Excel Macros in unattended mode on ScriptHost. Hence running Excel Macros using SQLPathFinder is discouraged and only supported for temporary legacy reasons until users can migrate there application to a more stable environment such as Python.

One alternative available to you is to add an Excel Macro which runs whenever a user opens Excel. This is described below:

The following example uses the Open event to run a macro when you open the workbook.

1. Open the workbook where you want to add the macro, or create a new workbook.
2. On the Developer tab, in the Code group, click Visual Basic.
3. In the Project Explorer window, right-click the ThisWorkbook object, and then click View Code.
Tip: If the Project Explorer window is not visible, on the View menu, click menu option *View->Project Explorer*.
4. In the Object list above the Code window, select Workbook.
5. This automatically creates an empty procedure for the Open event, such as this:

```
Private Sub Workbook_Open()
```

6. Add the following lines of code to the procedure:

```
Private Sub Workbook_Open()  
    MsgBox Date  
    Worksheets("Sheet1").Range("A1").Value = Date  
End Sub
```

7. Switch to Excel and save the workbook as a macro-enabled workbook (.xlsm).

8. Close and reopen the workbook. When you open the workbook again, Excel runs the Workbook_Open procedure, which displays today's date in a message box.

9. Click OK in the message box.

Note: The cell A1 on Sheet1 also contains the date as a result of running the Workbook_Open procedure.

SQLPathFinder currently allows you to load data to Excel and to optionally run an Excel macro. The former is fully supported. The latter action, however, of running an Excel macro is supported but discouraged for reasons stated above. Running a macro when the user opens the Excel workbook would be more in line with Microsoft's recommendation, as discussed above.

Note that if you just want to use SQLPathFinder to load one CSV/TAB file, you may use the default Excel Load Utility mode and just name the output Excel file. SQLPathFinder will create a sheet with the default name of SQLPathfinder. If you want to load more than one CSV/TAB file, you must use an existing Excel file (.xls, .xlsx, or .xlsm if the workbook contains macros to run) with an existing empty/named sheet for each CSV file. The latter mode also lets you run a macro (for xlsm files).

To do so, you could:

1. Name the Excel worksheet containing the SQLPathFinder data
2. Format the data in Excel as you want (e.g., add conditional formatting). You could also insert a new worksheet into the workbook and add a graph based on the original data. You could then create an Excel macro that simply refreshes the Graphs. You can use the Recording feature of Excel macros to automatically capture the VBA coding to refresh a Worksheet or Graph. Having set up the Workbook to do what you need, you could return to the original sheet with the SQLPathFinder data, clear the data, and save this Excel file. This now becomes your Excel **source or template** file.
3. After your SQLPathFinder main query on the SQLPathFinder Query Design window, you can add an Excel utility to the Tree to load the query output file to the appropriate Excel sheet in the Excel template file, to have SQLPathFinder run the macro to refresh the graphs or to apply formatting, and to save the Excel workbook to a new name in your working directory.

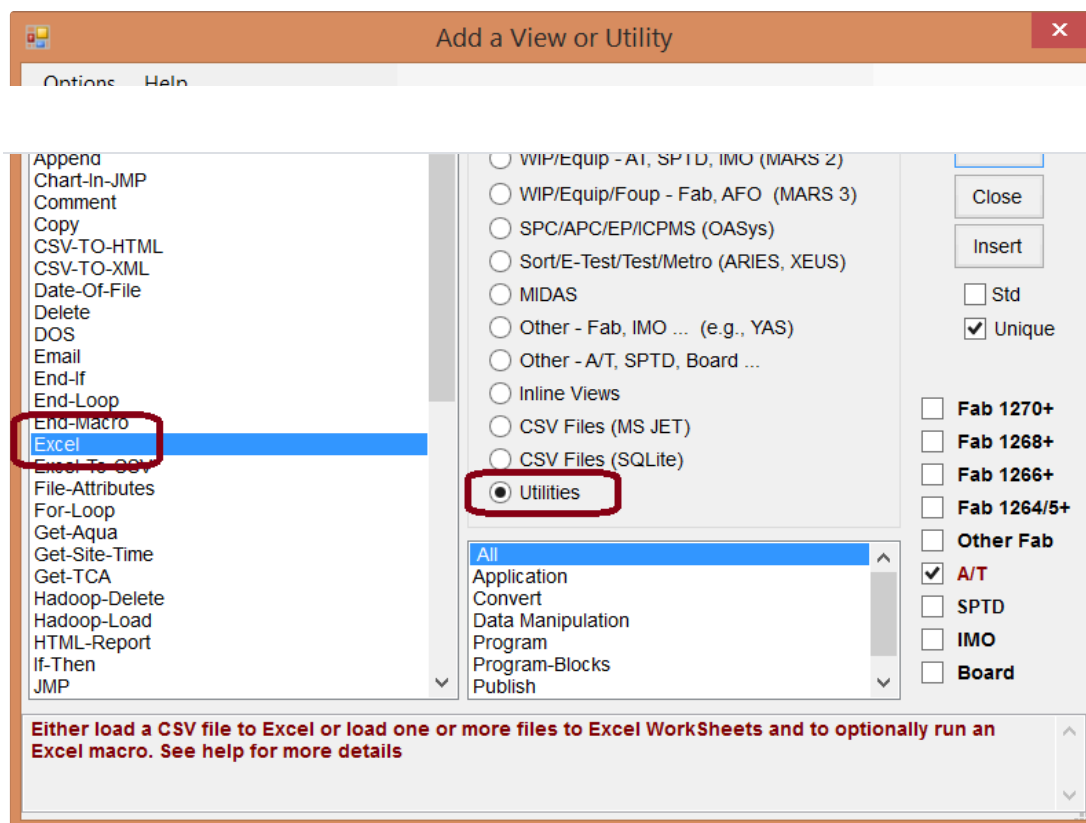
Here are some notes on doing this:

A) Adding an Excel Utility to Your Query

From the Query design form, choose menu option **Query -> Add a View or Utility**, select **Data Area Utility** on the **Add a View or Utility** window, and double-click the **Excel** item to add the **Excel Utility** to your query. See Figure 4 below. You can position the utility using drag-and-drop.

Double-click the **Excel Utility** in the Query Design Tree to configure it (See Figure 6)

Figure 4 - Adding an *Excel Utility* to Your Query



B) Overview of Excel Utilities

There are two Excel utilities, one to simply load a CSV file to Excel and another to load one or more Excel files to one or more Excel WorkSheets and to optionally run an Excel macro. The latter might include adding a plot or saving the output to HTML. Prior to using these Excel features you must configure your local Excel application to accept VBA jobs. To do so:

For Excel 2013:

- Start Excel
- Select menu **File -> Options**
- Select menu **Trust Center**, and button **Trust Center Settings**
- Select **Macro Settings** (See Figure 1 below)
- Check option **Trust access to the VBA project object module** under the **Developer Macro Settings** section. (See Figure 5 below)
- Click **OK**
- Close Excel

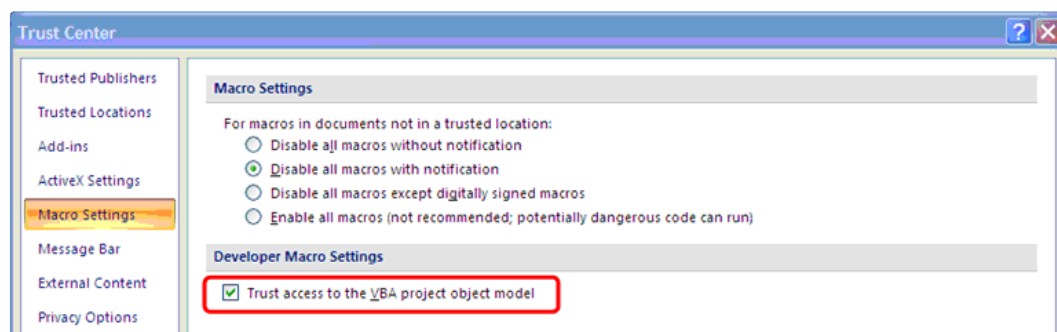
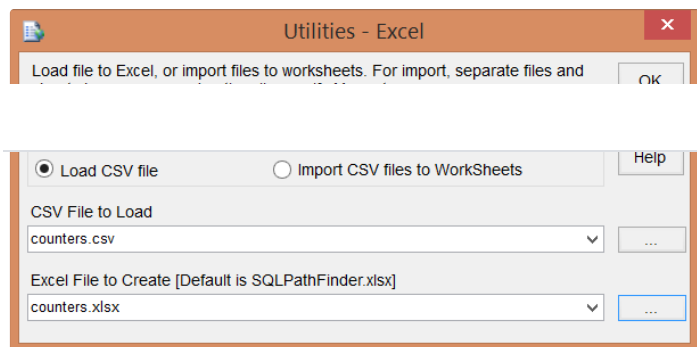


Figure 5 - Checking option "Trust access to the VBA project object module"

i) Loading a File to Excel

To import a file to Excel, ensure the **Load CSV file** option is checked on the **Utilities - Excel** form, and specify the name of the CSV file to import as well as the name and location of the Excel file to create. This is exemplified in Figure 6c below.



Utilities - Excel

Load file to Excel, or import files to worksheets. For import, separate files and sheets by commas, and optionally specify Macro to run.

☒ Load CSV file ☐ Import CSV files to WorkSheets

CSV File to Load: counters.csv

Excel File to Create [Default is SQLPathFinder.xlsx]: counters.xlsx

Figure 6c - Loading a CSV File to Excel

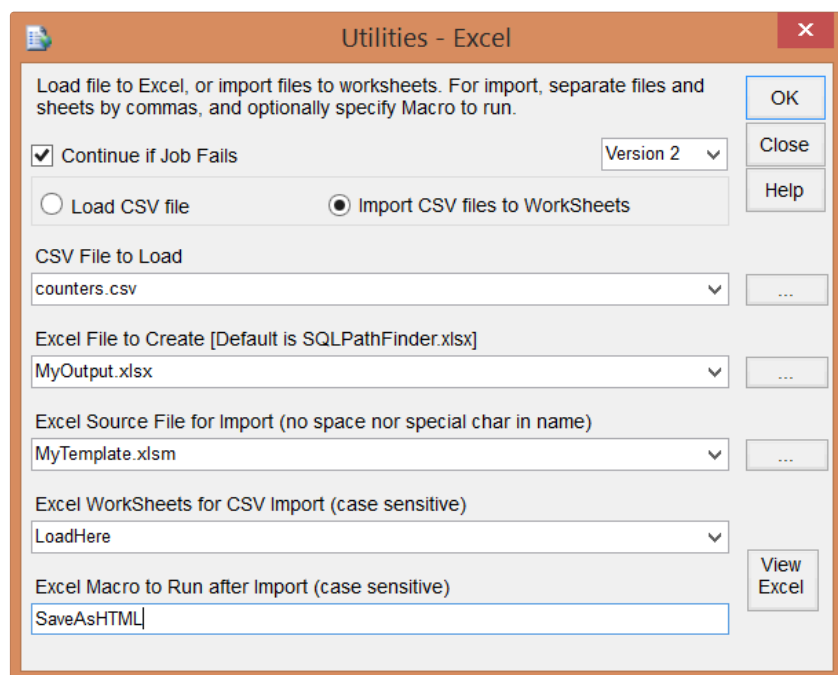
In this example, entry:

1. **Counters.csv** is the CSV file to load. It is located in the working directory, as no directory path is specified. For batch execution, the ScriptHost drone current directory represents the working directory.
2. **Counters.xlsx** is the Excel file to create. This will again be saved to the working directory as no file path is specified. This could also be some public share you have write access to, or that both you and ScriptHost have write access to.

ii) Importing a File to Excel and Running a Macro

One challenge with running Excel macros is that things do not work very well if the Excel file name or the Excel Macro name have spaces or special characters. As such, please avoid adding spaces or special characters in these Excel file and macro names.

To import a CSV file to Excel and run a macro, **check** option **Import CSV files to WorkSheets**, and configure the Utilities form as exemplified in Figure 7.



Utilities - Excel

Load file to Excel, or import files to worksheets. For import, separate files and sheets by commas, and optionally specify Macro to run.

☒ Continue if Job Fails Version 2

☐ Load CSV file ☒ Import CSV files to WorkSheets

CSV File to Load: counters.csv

Excel File to Create [Default is SQLPathFinder.xlsx]: MyOutput.xlsx

Excel Source File for Import (no space nor special char in name): MyTemplate.xlsm

Excel WorkSheets for CSV Import (case sensitive): LoadHere

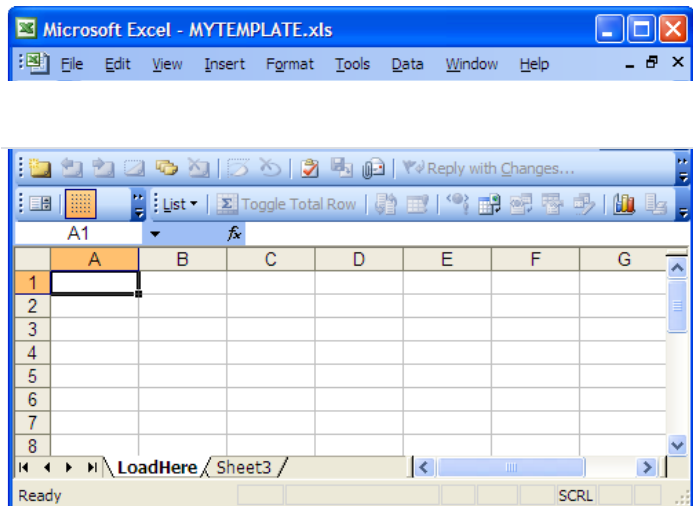
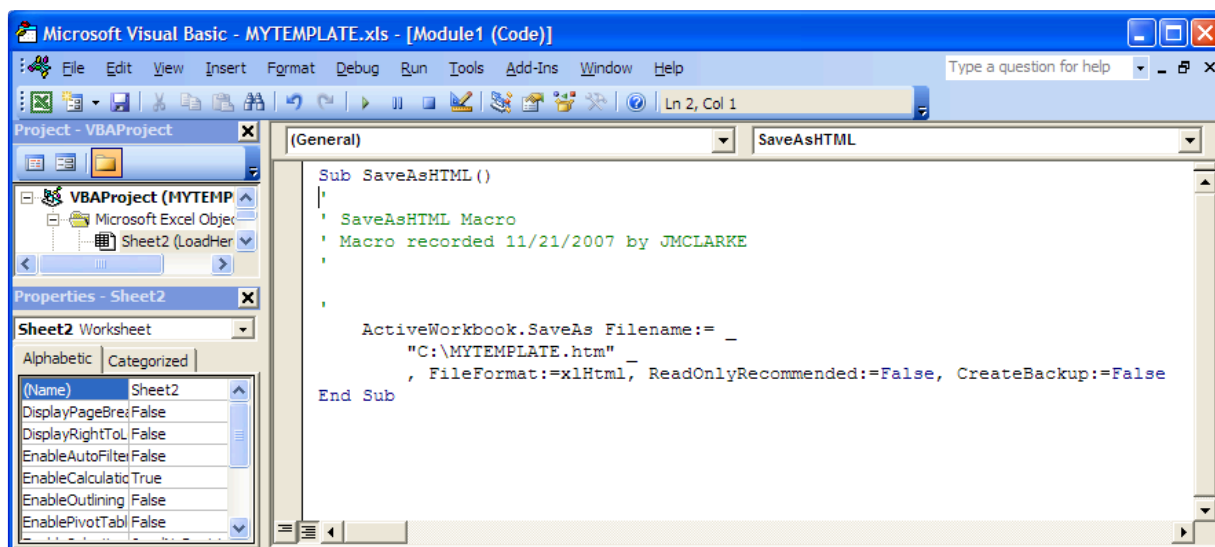
Excel Macro to Run after Import (case sensitive): SaveAsHTML

Figure 7 - Excel Form

In this example, entry:

1. **Counters.csv** is the CSV file to load. It is located in the working directory, as no directory path is specified
2. **MyOutput.xlsx** is the Excel file to create. This will again be saved to the working directory as no file path is specified.
3. **MyTemplate.xlsm** is the Excel template containing the macro to run. This file has a macro called **SaveAsHTML**. **Ensure the Template file does not have spaces in its name, nor any special characters such as parentheses. This can cause the macro to fail.**
4. **LoadHere** represents the WorkSheet in Excel file **MyTemplate.xlsm** where file **counters.csv** will be loaded. Note that Sheet names are case sensitive.
5. **SaveAsHTML** is the macro in Excel file **MyTemplate.xlsm** that is to be run. Note that **MyTemplate.XLSM** is read only and not updated. File **MyOutput.xlsx** is created by this routine based on **MyTemplate.xlsm**. **Note that Macro names are case sensitive..**

The **SaveAsHTML** macro in this example simply saves an HTML version of **MyOutput.xlsx** after the file was loaded. Sample images of the contents of **MyTemplate.xlsm** are shown in Figures 8 and 9.

Figure 8 - *MyTemplate.xlsm* Showing the *LoadHere* Worksheet where Data is to be LoadedFigure 9 - Macro *SaveAsHTML* Created as a Recording in *MyTemplate.xlsm*

C) Special Note for When Running the Import Excel Utility in Batch

To run the Excel Import utility in batch on Script Host, the Excel Template file must be placed in a public share that can be read by Script Host. All SQLPathFinder users should have a personal ScriptHost share of format: `\\shuser-prodat.intel.com\SHProdATUser$\<windows user name>\` in the Assembly/Test ScriptHost farm or `\\shuser-prod.intel.com\SHProdUser$\<windows user name>\` in the Fab ScriptHost farm. A revised SQLPathFinder *Utilities - Excel* form for this case is shown in Figure 10.

Figure 10 - For Batch Runs, place Template File in your Personal ScriptHost Directory

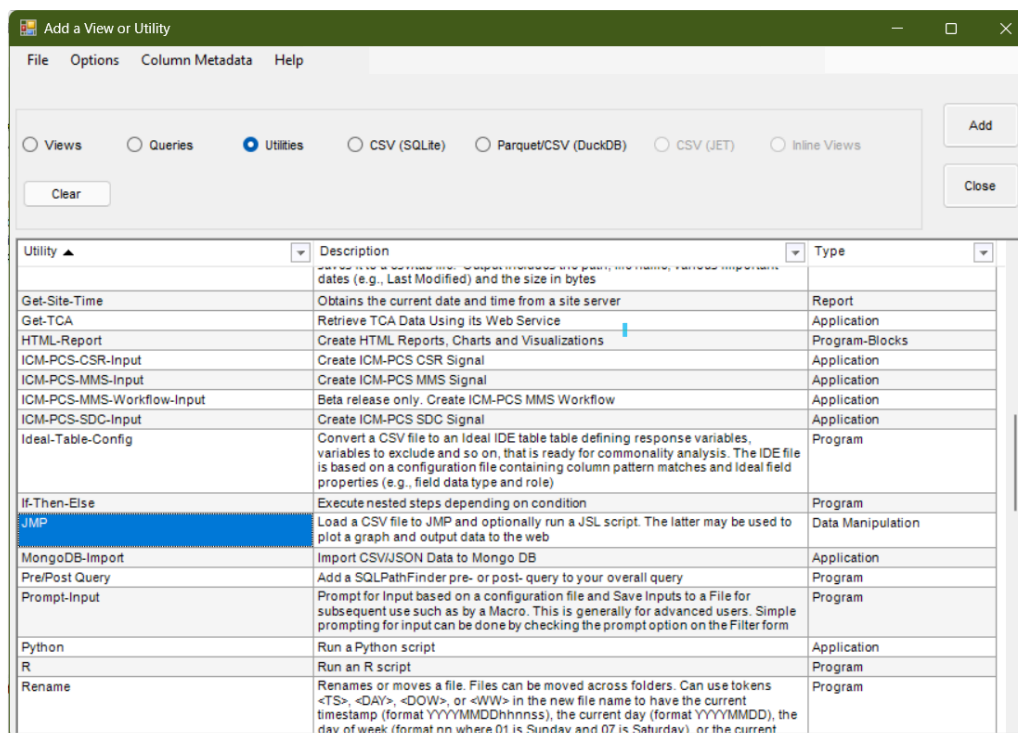
[\[Return to Top\]](#)

JMP Utility - Formatting Output or Plotting Charts

[\[Return to Top\]](#)

For enhanced integration of SQLPathFinder with JMP/JSL, [see this topic](#). SQLPathFinder also allows you load a CSV file to JMP or to execute a JSL script. The latter may be used to plot a graph and output data to the web following a query run. To add a JMP utility to your query, choose menu option **Query -> Add View or Utility**. Then select **Data Area Utility** on the *Add a View or Utility* window, and double-click the JMP item to add a JMP *Utility* to your query. See Figure 11 below. You can then position the utility using drag-and-drop.

Figure 11 - Adding a JMP *Utility* to Your Query



Double-click the JMP *Utility* in the Query Design Tree control to set it as suggested in Figure 13.

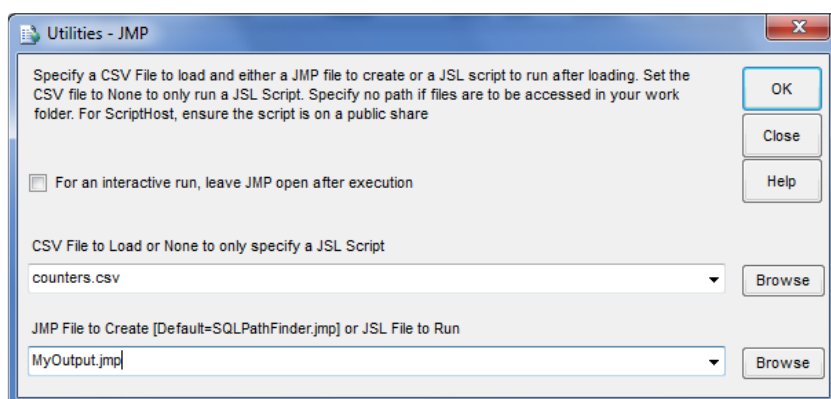


Figure 13 - The **Utilities- JMP** Form

In Figure 13:

1. **Counters.csv** is the CSV file to load. It is located in the working directory, as no directory path is specified. For batch execution, the default folder on the ScriptHost drone running the query represents the working directory.
2. **MyOutput.jmp** is the JMP file to create. This will again be saved to the working directory as no file path is specified.

Instead of a JMP file, the path and name of a JSL script can also be specified, as illustrated in Figure 14. For this case, SQLPathFinder will not load any file to JMP but will simply run JSL script `JSLTest.jsl` located in public share `\\azsactapp22\scripting\library\ymclarke\JSL`. For ScriptHost use, the JSL script must either be copied to the default folder as part of the SQLPathFinder job or referenced on a public share.

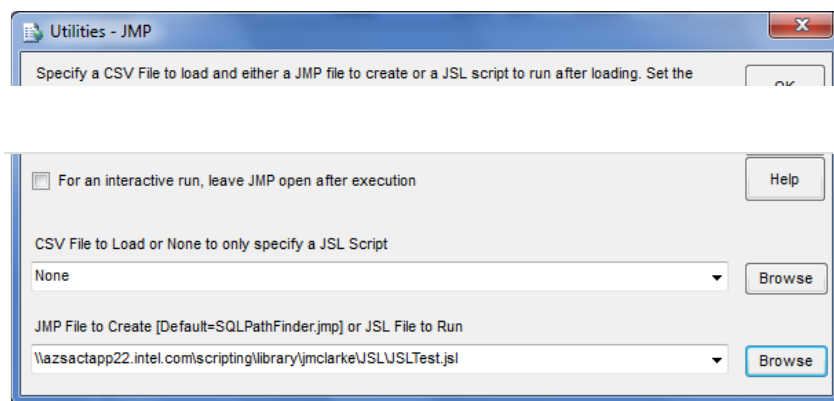


Figure 14 - Running a JSL Script

Should you use SQLPathFinder's ability to load a CSV file and also run a JMP->JSL script, the SQLPathFinder utility will load the CSV or TAB (i.e., Comma or TAB Separated Values) file, reference the new JMP Data set by variable **csv1**, and set global variable **Default_Dir** to point to the default folder where the job is running whether the job is run interactively or on a ScriptHost drone. Of course, if you execute JMP command `Clear Globals()`, this variable will be erased. You can reference the **csv1** and the **default_dir** variables in your script if needed.

An example of a reference in a JSL Script might be:

```
RChart = Chart(X(:ENTITY), Y(N(:ENTITY_AVAILABILITY)), Bar Chart(1));
RChart<<Save RTF("\\server\path\JSLTest.rtf");

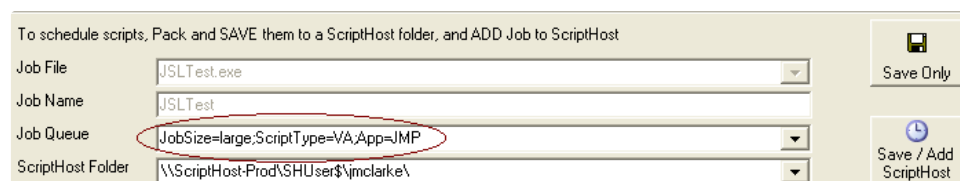
OutPath = Concat(Default_Dir, "JSLTest.htm");
csv1<<save html(OutPath);

close(csv1, NoSave);
```

Special Note for When Running the JMP Utility in Batch

When running a JSL utility in batch, it is important to place the JSL script in a public share that ScriptHost can access. This is exemplified in Figure 14.

In addition, when configuring the *Process Multiple Queries* form to run a batch job which includes the JMP utility, ensure the job queue has the string **App=JMP** appended to it, as illustrated in Figure 15. This ensures that the batch server has JMP installed. If this is not done the batch job may fail when executing the JMP utility.

Figure 15 - Including the **App=JMP** Clause in the Job Queue Name, in order to run a JMP Utility in Batch on Script Host

Debugging JMP scripts can, however, be challenging on ScriptHost. SQLpathfinder does print the JMP log at the end of the JMP utility. Hence, if you add any print statements, e.g.,

```
print ("Loaded data and starting analysis ...");
```

...these print statements will show up in the log to perhaps assist you to determine where a particular issue occurred, if the default error message in the JMP log is otherwise not helpful. As JMP 9 and higher is also multi-threaded, you may need to add a **Wait-Utility** to wait for the JMP output to be created before moving onto the next SQLPathFinder Job step if that step is dependent on the JMP output.

Finally, use option **"For an Interactive run, leave JMP open after execution"** on the JMP utility form with care. This option is intended for use should you already have JMP open when you run this utility, and you do not want SQLPathFinder to close the previously open JMP session. You could end up with invisible JMP sessions if this option is checked and JMP was not previously started. Some people find using the **Chart-in-JMP** utility a little easier for interactive use.

Optimization Tips on Using JMP:

Processing large files remotely in JMP introduces network and timeout errors. As such:

Make sure you are processing data locally using JMP. I.e., ensure an output file being read into JMP is in the local SQLPathFinder (SPF) work folder and not on a remote share (i.e., a share starting with \\). For such cases, no path is specified for the file, e.g., input file may be **abc.tab**.

Make sure you save the file created by JMP to the local work folder and not to a remote share. You can always add an SPF **Copy** utility to copy the journal or JMP or CSV file created by JMP to its final destination after the JMP utility runs. As JMP often releases control, continues on in the background, and goes on to the next step before JMP has completed a particular step, you will need to add an SPF **Wait-File** utility after the JMP utility to wait until the JMP Journal or other file is created before the Copy utility runs, otherwise the copy utility may not find the file to copy.

When loading data to JMP, specify column data types versus letting JMP decide on default datatypes. This can significantly speed up the load of the file to JMP. If datatypes are not specified, JMP reads the input file twice, once to determine datatypes and then to load the data.

E.g., of inclusion of datatypes in a JMP load is as follows:

```
Open ( "abc.tab",
columns( DEFECT_ID = Numeric,
         INSPECT_EQUIP_ID = Character,
         INSPECTION_DATE = Numeric,
         KEYSORT = Character,
         LOT_ID = Character ),
```

```

Import Settings(
    End Of Line( CRLF, CR, LF ),
    End Of Field( Comma ),

    Treat empty columns as numeric( 0 ),
    Labels( 1 ),
    Column Names Start( 1 ),
    Data Starts( 2 ),
    Lines To Read( All ),
    Year Rule( "10-90" )

);

```

[\[Return to Top\]](#)

Emailing a File to a Distribution list

SQLPathFinder allows you to email results to a distribution list. To add an Email utility to your query, choose menu option **Query -> Add Table or Utility** from the Query design form. Then select Data Area *Utility* on the **Add a View or Utility** window, and double-click the Email item to add an Email Utility to your query. See Figure below. You can then position the utility using drag-and-drop.

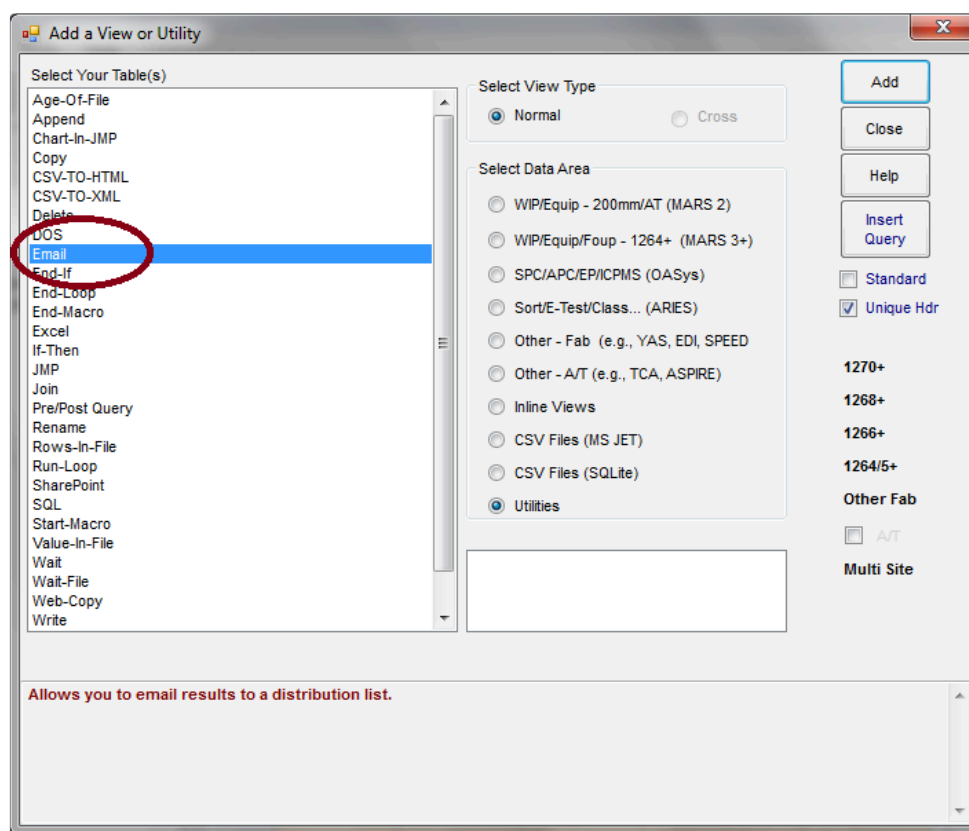


Figure - Adding an Email Utility to your Query

Double-clicking the Email *Utility* in the Query Design Tree invokes the *Utilities - Email* form as shown in the Figure below.

Specify the attachment files to email in the **Optional File(s) separated by commas...** text box. You can enter multiple files separated by commas. You can also specify an email distribution list in the **Specify an email distribution...** text box. The token **self** will cause the file to be emailed to you. Separate other email addresses with commas. E.g., **self, joan.doe@intel.com** will email the file(s) to you and to Joan Doe. Instead of specifying a comma delimited list of addresses, you may also reference an ANSI text file containing a column header and a list of addresses with one line per address. E.g.,

```
Email_Addresses
John.Doe@Intel.com
Jane.Doe@Intel.com
```

In addition, you can also optionally reference a file containing the email message body. The file name and path should be entered in the **Optionally File Containing text for message body** text box. For jobs run in batch, this file should be located on some public share that you have read access to. For a straight text file, the format of the ANSI text file (e.g., spaces or line breaks,...) will be preserved. You can also send a file with HTML formatting. If the file contains the tags **<HTML>** and **</HTML>**, SQLPathFinder will interpret it as an HTML file. E.g.,

```
<HTML>
<BODY>
VIA FCCD Matching Report for the last 7 days.
<br>

<br>

</BODY>
</HTML>
```

You can also specify an Email subject in text box **Optional subject line...**. Click **OK** to commit your changes, or click **Close** to cancel changes and leave the email definition unchanged.

Finally, on the email utility, note that there is a text box: **Rialto Roles separated by commas**. This option will only mail addresses passed to it in a file that have the particular list of Rialto roles. The option requires that you pass it a list of email addresses in a comma or tab delimited file. An example of the email address file for use with Rialto role checking is shown below:

```
email, domain, idsid
john.doe@intel.com, amr, jdoe
```

Publishing a File to SharePoint

[\[Return to Top\]](#)

SQLPathFinder allows you to publish a file to a Sharepoint site. To add a SharePoint utility to your query, choose menu option **Query -> Add Table or Utility** from the Query design form. Then select **Data Area Utility** on the **Add a Table** window, and double-click the SharePoint item to add a **SharePoint Utility** to your query. See Figure 16 below. You can then position the utility using drag-and-drop.

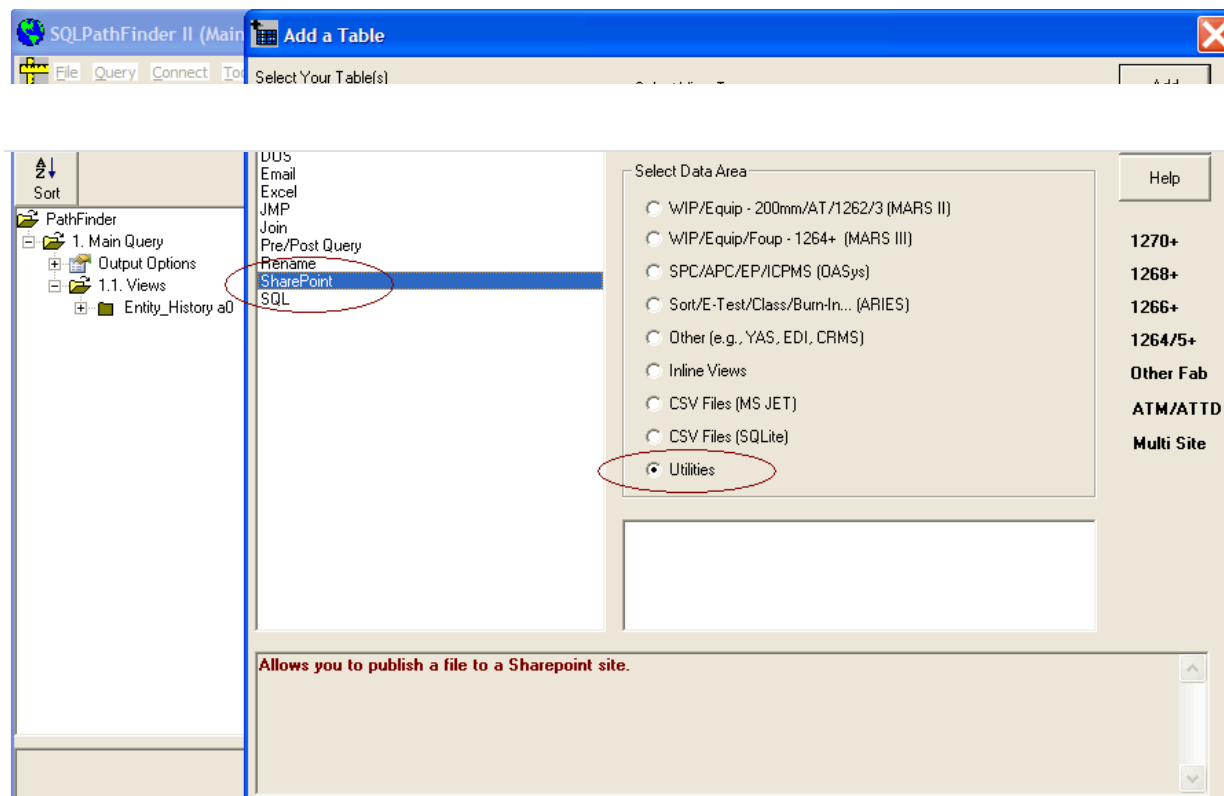


Figure 16 - Adding a SharePoint Utility to Your Query

Double-clicking the SharePoint Utility in the Query Design Tree invokes the *Utilities - Sharepoint* form as shown in Figure 18. Specify the file to publish in the **File to publish...** text box. Specify the web address to publish the file to in the **Specify a web address ...** text box. Make sure that you include the destination file name at the end of the web address as illustrated in Figure 18. Click **OK** to commit your changes, or click **Close** to cancel changes and leave the query unchanged.

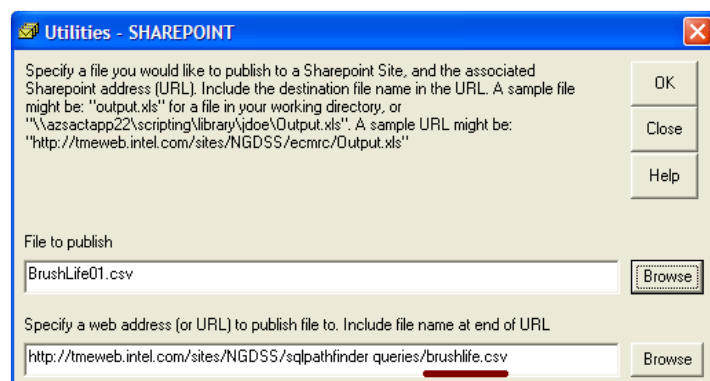


Figure 18 - The Utilities - SHAREPOINT Form

[\[Return to Top\]](#)

Application of the If-Then and the Count Rows in File Utilities to Conditionally Execute Steps Depending on File Existence

[\[Return to Top\]](#)

The **If-Then-Else** utility allows you to conditionally execute steps depending on the value of a Variable. The **Count Rows in File** utility allows you to count the number of rows in a file and assign the row count to an environment variable. The combination of the two utilities allows you to conditionally execute steps depending on whether a file exists or based on the number of rows in a file. An example where an email utility is conditionally executed depending on whether a file exists and contains more than 0 rows is described below.

Start by adding and ordering a Rows-In-File, an If-Then-Else, and an Email utility to your query as illustrated in Figure 19.

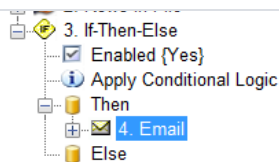
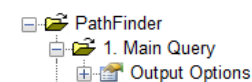


Figure 19 - Conditionally Emailing an Output File

Double-clicking the **Rows in File** utility on the Tree control invokes the *Rows-In-File utility* as shown in Figure 20. On this form, you can browse for a file to check the row count and the name of the Environment Variable to hold the number of rows. This environment variable can then be referenced in other steps of the *query*. In Figure 20, the number of rows in file **Lot_Status.csv**, which is located in the working directory, is computed and the row count is written to environment variable **RowsInFile**. If the file cannot be found or there is a problem accessing the file, then a value of -1 will be assigned to the environment variable. Hence a later test if the environment variable is greater than 0 will determine if the file contains at least one data row. The file being tested is assumed to have a header row, and the row count actually excludes the first row in the file.

Utilities - ROWS-IN-FILE

Computes number of data rows in a file (excluding header) and assigns count to an Environment Variable. Assigns -1 if file is not found, and 0 if file only contains a header. Check option below to stop counting if there is any data in file

☒ Stop Counting After First Data Row

Input CSV File to check
Lot_Status.csv

Environment Variable to Assign Row Count
RowsInFile

OK Close Help

Figure 20 - The Rows in File Utility Form

Double-clicking the **If-Then-Else** utility invokes the *IF - THEN Utilities* dialog box as shown in Figure 21. Here one enters the name of variable to test, the conditional operator and the conditional value. In Figure 21, a test on whether environment variable **RowsInFile** is greater than **0** will be applied. If the condition is true, then all **"THEN"** steps are executed.. The conditional step, as illustrated in Figure 19, is step 4 which is to email the contents of the query output file to a distribution list. As such, the emailing will only be done if the file exists and contains at least one row of data.

Utilities - IF-THEN

If condition-1 and/or optional condition-2 are true, execute nested steps. Compare Operators ending in S (e.g., EQS) do string compares. (E.g., 2 LT 11 but '2' GTS '11'). To compare environment vars, enter ENV(name) as Compare Value

Environment Var or Global Var-1 [VAR(<<<var>>>)]
RowsInFile

Compare Operator-1
GT

Compare Value-1
0

AND/OR

Environment Var or Global Var-2 [VAR(<<<var>>>)]

Compare Operator-2

Compare Value-2

OK Close Help

Figure 21 - The IF-THEN-ELSE Utility Form

For the If-Then-Else form, potential conditional operators include:

Operator	Description
EQ	Test for equality on numbers

NE	Test for not equal to on numbers
NES	Test for not equal to on strings
GT	Test for greater than on numbers
GTS	Test for greater than on strings
LT	Test for less than on numbers
LTS	Test for less than on strings
GE	Test for greater than or equal to on numbers
GES	Test for greater than or equal to on strings
LE	Test for less than or equal to on numbers
LES	Test for less than or equal to on strings

[\[Return to Top\]](#)

Appending Data to a File, Deleting a File, Renaming a File, or Moving a File

[\[Return to Top\]](#)

SQLPathFinder allows you to manipulate files required by or generated by your query. For example, you can append data to a particular file, delete one or more temporary files, or rename a file and incorporate tokens such as the current work week, current day, or current timestamp into the file name.

a. Adding and Positioning Utilities

To add a utility to your Query, from the Query design form choose menu option **Query -> Add Table or Utility**, and select **Data Area Utility** on the **Add a Table** window. You will be shown a list of utilities as illustrated on Figure 22. To add a utility, double-click the utility name. The utility will then be added to the end of your query as illustrated in Figure 23. Close the Add a Table window to interact with the utility. To position a utility, use drag-and-drop.

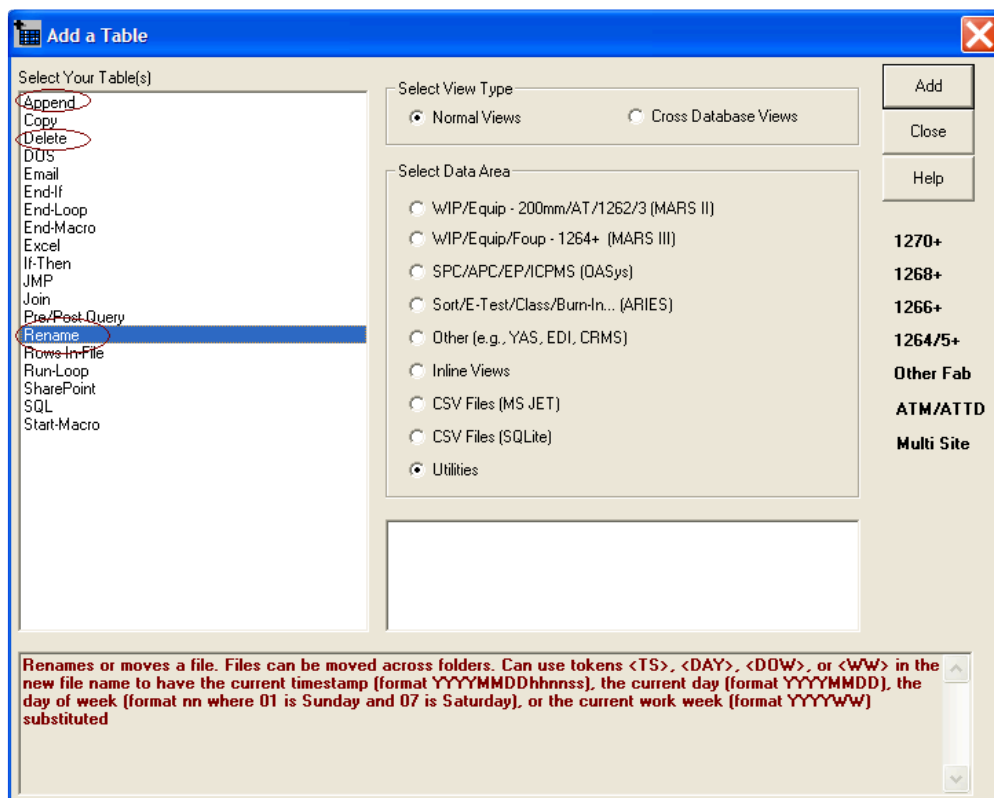


Figure 22 - Adding a Utility to Your Query

b. Appending Data

SQLPathFinder allows you to append output results to another file. SQLPathFinder also allows you to limit the number of rows in the final file. The latter can be useful if the final application you wish to load data to, e.g., Microsoft Excel, has built in row constraints. SQLPathFinder will eliminate rows from the final file starting at the top of the final file.

To utilize this utility, double-click the utility in the Tree Control of the Query design form and fill in the **Utilities - APPEND** form as suggested in Figure 24. This form requires a destination file which will receive the rows and a source file. If no directory paths are specified, the source and destination file will be assumed to be in your local or ScriptHost working directories. If the destination file (or file to have rows appended to) does not exist, SQLPathFinder will create it using the source file. You can also specify the maximum number of rows allowed in the destination file. SQLPathFinder will delete rows starting from the top of the final file such that your maximum row count

Figure 24 - The Append File Utility Form

c. Deleting a File

SQLPathFinder allows you to delete intermediate files created by your job or otherwise. To utilize this utility, double-click the utility in the Tree Control of the Query design form and fill in the **Utilities - DELETE** form as suggested in Figure 25. This form requires a directory and file name (with optional wildcards) to delete. E.g., Poly*.csv. If there is no destination path, the file(s) will be assumed to be in your default directory which should map to your local SQLPathFinder working directory, or your ScriptHost working directory for jobs run on ScriptHost. Set second argument to Y to not have to confirm file deletions when *.* is specified or to automatically delete Read-Only files. Click OK to commit your changes and return to the Query design form.

Figure 25 - The Delete File Utility Form

d. Renaming or Moving a File

SQLPathFinder allows you to rename or move files created by your job or otherwise. To utilize this utility, double-click the utility in the Tree Control of the Query design form and fill in the **Utilities - RENAME-MOVE** form as suggested in Figure 26. Specify a file to rename or move as well as the new file name or folder. Specify no path if the file is in your local or ScriptHost working directory. Tokens <TS>, <DAY>, <DOW>, and <WW> in the new name will be replaced with the current timestamp in format YYYYMMDDHHnnss, the current day in format YYYYMMDD, the day of week in format nn with 01 being Sunday and 07 being Saturday, and the current work week in format YYYYWW respectively. In Figure 25, file Sort_YAS.csv located in the work directory will be renamed to Sort_YAS_20090808.csv if this were August the 8th of 2009. Click **OK** to commit your changes and return to the Query design form.

Figure 26 - The Rename-Move File Utility Form

[\[Return to Top\]](#)

Application of the If-Then-Else and Value-In-File Utilities to Compare a User Name in a File to the "UserName" Environment Variable

Say in your group you have critical jobs and you need coverage in the event a person running a critical job goes on vacation. You could have two users schedule the same job on ScriptHost, but use a configuration file to determine which user will actually run the job. If you place the active username in a configuration file (i.e., a comma or Tab separated text file with column headers as illustrated in Figure 29), then you can use the SQLPathFinder **Value-In-File** and **If-Then-Else** utilities to test for that name and compare it to the user running the current job (i.e., compare it to the system environment variable USERNAME). As such only one user job would actually run and the other user job would simply end quickly after it determines it is not the active user.

Consider the following flow as shown in Figure 27. A **Value-In-File** Utility is used to test for the value of column **UserName** in File **MyConfiguration.csv**, and to assign the name to environment variable **MyUserName**. The setting of the **Value-In-File** utility, obtained by double-clicking the object, are shown in Figure 28. The contents of file **MyConfiguration.csv** are shown in Figure 29. The **If-Then** utility is then used to compare the value of environment variable **MyUserName** with the system environment variable **USERNAME** as shown in Figure 30. Note that wrapping an environment variable in the **ENV()** expression (e.g., **ENV(UserName)**) returns the actual value of an environment variable.

If the condition is true, the Main Query will be executed per the flow shown in Figure 27. A sample log of such a job is shown in Figure 31 below.

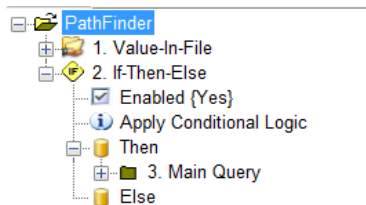


Figure 27 - Application of the Value in File and If-Then Utilities

Utilities - VALUE-IN-FILE

The (VALUE-IN-FILE) utility retrieves the value of a named column in row 1 of a named file and assigns it to an environment variable. If the file or column are missing, a value of ERROR is returned

Input CSV File to Check: MyConfiguration.csv [Browse]

Environment Variable to Assign: MyUserName [v]

Column Header in File to Access: UserName [v] [Refresh]

[OK] [Close] [Help]

Figure 28 - Settings for the Value-In-File Utility

UserName
JMCLARKE

Figure 29 - Sample Contents of File *MyConfiguration.csv*

Utilities - IF-THEN

The (IF-THEN) utility will execute all steps until the associated (END-IF) token is reached, provided the specified environment variable condition is true. Comparison operators end in S (e.g., EQS) if a string comparison is to be done. Otherwise a numeric comparison will be done. Embed environment variables to compare with ENV() with no spaces. E.g., DataInFile EQS ENV(UserName)

Environment Variable: MyUserName

Comparison Operator: EQS [v]

Comparison Value: ENV(UserName) [v]

[OK] [Close] [Help]

Figure 30 - Settings for the If-Then Utility

Starting script v2.9 ... 29-Aug-2010 11:21:21

File to process: MyConfiguration.csv

Assigning value of column USERNAME for row 1 to MYUSERNAME
Value is JMCLARKE

Connecting to .\
Done 29-Aug-2010 11:21:21

Fetching Text Data ... 29-Aug-2010 11:21:21
Done. 3356 row(s) returned... 29-Aug-2010 11:21:22

If Statement Completed ...

SQLP>

Figure 31 - Log of a Sample Job that Uses Value-In-File and If-Then

[\[Return to Top\]](#)

Determining the Age of a File as a Precursor to some action

[\[Return to Top\]](#)

Say you wished to delete a file if it were older than 48 hours. The **Age-Of-File** utility allows you to determine the age of the file, computed as the number of hours, days, minutes, or seconds between when the file was last modified and right now, and to assign the number to an environment variable. This could be combined with the **If-Then** utility to conditionally test for the file age. Such a flow is illustrated in Figure 32. The settings for the **Age-Of-File** utility are shown in Figure 33. File **Test.csv** is accessed and its age in **hours** in this case is assigned to environment variable **AgeOfFile**. An **If-Then** utility is then executed to determine the age of the file. If the age exceeds 48 hours the file is deleted. The settings for the **If-Then** and the **Delete** utilities are shown in Figures 34 and 35 respectively. A sample log of the job is shown in Figure 36.

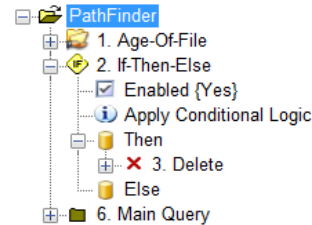


Figure 32 - Application of the Age-Of-File and If-Then Utilities

Figure 33 - Settings of the Age-Of-File Utility



Figure 34 - Settings for the If-Then Utility

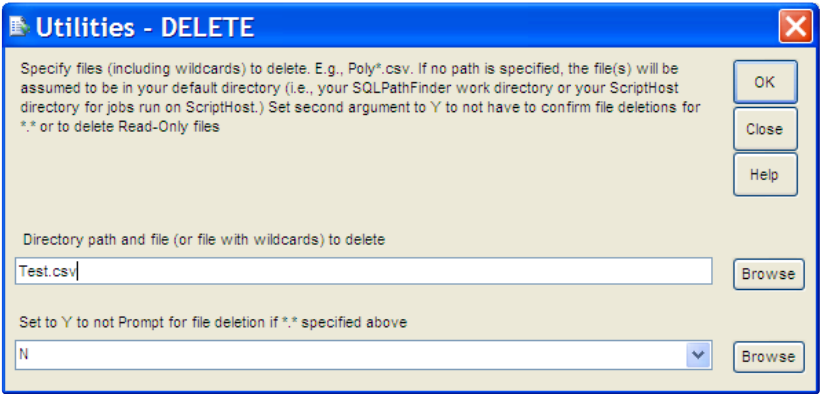


Figure 35 - Settings for the Delete Utility

```
Starting script v2.9 ... 29-Aug-2010 11:40:22

File to process: Test.csv
Assigning Age of File to: AGEOFFILE
Units of Time for Age will be: Hours
  File was last modified on 23-Aug-2010 15:09:50
  File is 140 Hours old...

Starting IF-THEN Logic ...
Comparing ( AGEOFFILE = 140 ) GT 48
If Condition is True...

Starting Delete Applet, v2.0 ... 29-Aug-2010 11:40:22

Deleting Test.csv
Done ...

If Statement Completed ...

SQLP>
```

Figure 36 - Sample Log of the Job Involving Age-Of-File, If-Then, and Delete Utilities

[\[Return to Top\]](#)

Smart Append of Data to a File

[\[Return to Top\]](#)

SQLPathFinder helps you to intelligently append data to an existing file. This includes appending files with varying columns or columns in varying orders, as well as removing expired data from a file or updating rows in a file based on a key lookup. To add the Smart Append utility to your query, choose menu option **Query -> Add Table or Utility** from the Query design form. Then select Data Area -> **Utilities** on the **Add a View or Utility** window, and double-click the *Smart Append* item. This is illustrated in Figure SA1 below. You can then position the utility in your query using drag-and-drop.

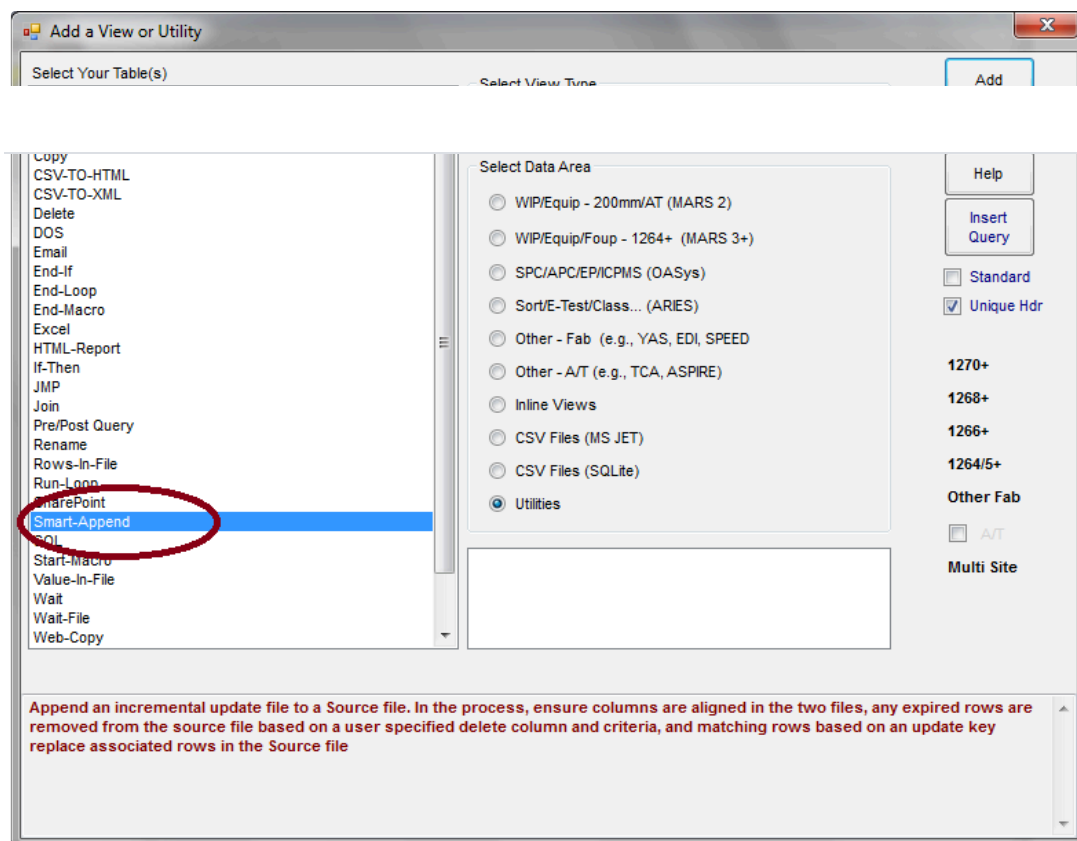


Figure SA1 - Adding a Smart-Append Utility to Your Query

To configure the SMART Append utility, double-click the utility on the query form. This invokes the *SMART-APPEND* form as illustrated in Figure SA2. Specify the old file which is to be updated in the *File with old data...* text box. If the old file does not exist, it will be created from a copy of the new file. Specify the incremental file that inserts, deletes, or updates rows in the old file in the *File with new data...* Text box. Specify the column that flags rows for deletion in the old file in the third text box. If this text box is left empty, no rows will be deleted from the old File. Specify the delete criteria in the fourth text box. **All rows in the old file as well as the new file with values in the "delete column" that are less than the delete criteria will be removed. Note that rows that equal to the delete criteria will not be removed by design.** In addition the delete check and criteria is **case insensitive**. In Figure SA2 below, rows will be removed from the old file if column "*Txn_Date*" is less than **20110102**.

You may also specify the column to be used to determine if rows should be updated in the old file in the fifth text box on the *Smart Append Utility* form. Any rows with the same key value in the old file as in the new file will be replaced in the old file. In Figure SA2, column **Lot** is expected to be in both the old and the new file. Any rows in the new File with the same lot number as the old file, will override that row in the old file. All remaining rows in the new file will be inserted into the old file.

By default, columns in the old file will be sorted alphabetically after processing. However, if you wish to preserve the original column order, uncheck option "**Sort Column Headers in the Final Output**" in the *Smart-Append* Utility form. With this unchecked, the column order of the old file will be preserved, and any new columns will be added to the end of the file.

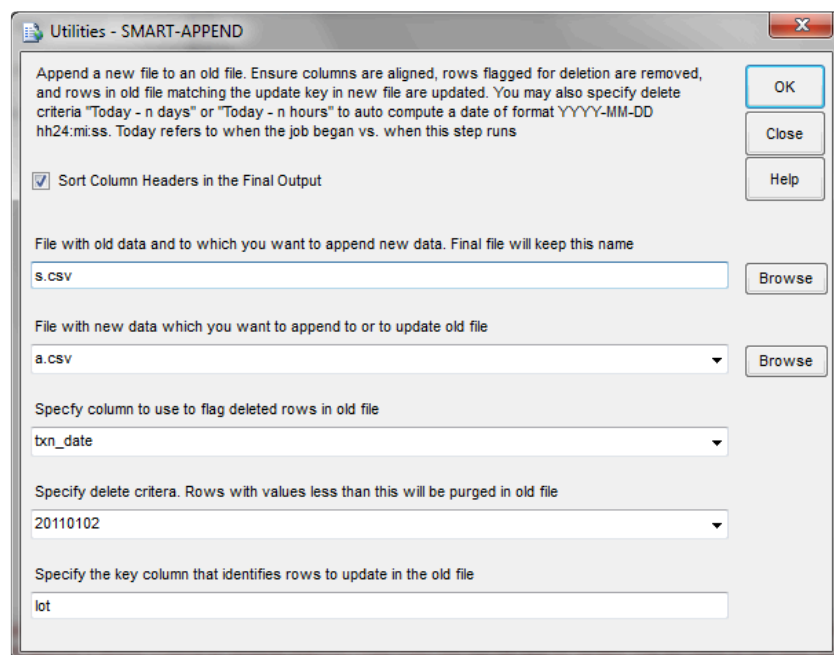


Figure SA2 - Configuring the SMART-Append Utility

A sample use case is provided below for illustrative purposes. Figure SA3 shows a sample comma or Tab delimited *old* File containing five columns and five rows. Figure SA4 shows a sample new file. The following will occur after applying the Smart-Append configuration shown in Figure SA2:

- Rows for lots **12** and **15** will be deleted from the old file as their **txn_date** column values are less than or equal to the delete criteria of **20110102**. Note that the data filter method is alphanumeric vs. numeric. Note that you can also use tokens "**Today - n days**" and "**Today - n hours**" to automatically compute dates. A delete

criteria of **Today - n days** or **Today - n hours** will compute a date and time that is n days or n hours respectively from midnight on the day the job began as opposed to when this utility was run in the event the overall job spans more than one day.

- The row for lot **I12** in the new file will also be removed as it is older than the expired date.
- As column **I4** is specified as the update key and lots equal to **I4** and **I3** match in both files, lots **I4** and **I3** from the new file will replace matching rows in the old file.

old file. If checkbox *Sort Column Headers in the Final Output* was unchecked in the Smart Utility Append, the resulting file would be as shown in Figure SA6.

lot	operation	entity	txn_date	newcol1
I1	o11	e11	20110404	a1
I2	o2	e2	20110101	b1
I3	o31	e31	20110404	b21
I4	o4	e4	20110501	c1
I5	o5	e5	20110101	d1

Figure SA3 - Original (or Old) File

lot	operation	entity	txn_date	newcol2
I1	o11	e11	20110501	a2
I3	o31	e31	20110501	b2
I8	o4	e4	20110401	c2
I9	o5	e5	20110501	d2
I10	o6	e6	20110601	e2
I11	o7	e7	20110701	f2
I12	o8	e8	20100101	g2

Figure SA4 - New (or Append) File

entity	lot	newcol1	newcol2	operation	txn_date
e4	I4	c1		o4	20110501
e11	I1		a2	o11	20110501
e31	I3		b2	o31	20110501
e4	I8		c2	o4	20110401
e5	I9		d2	o5	20110501
e6	I10		e2	o6	20110601
e7	I11		f2	o7	20110701

Figure SA5 - Final Output After the Smart Append Utility

lot	operation	entity	txn_date	newcol1	newcol2
I4	o4	e4	20110501	c1	
I1	o11	e11	20110501		a2
I3	o31	e31	20110501		b2
I8	o4	e4	20110401		c2
I9	o5	e5	20110501		d2
I10	o6	e6	20110601		e2
I11	o7	e7	20110701		f2

Figure SA6 - Final Output If Option "*Sort Column Headers in the Final Output*" was unchecked on the *Smart-Append* Utility Form

Smart-Append Version 2 and 3

Smart-Append version 2 leverages SQLite for the Smart-Append capability as opposed to VA used in Smart-Append Version 1. Smart-Append Version 2 is faster and can handle larger file sizes. Only limitation with Smart-Append Version 2 is that it can only handle up to 30,000 columns. You should always try to use Smart-Append Version 2.

Smart-Append version 3 builds upon Smart-Append Version 2. The goal of the new release is to add additional steps to reduce the chances of a corrupted final file. The below new capabilities will be available to you if you utilize Smart Append Version 3. Jobs using Smart Append Version 1 or Version 2 will not be affected. The additional Smart-Append steps added as part of Version 3 are as follows.

- It optionally allows you to abort the job if the Old File is not found.

- It will abort the job if the New file is stored remotely which could yield performance problems.

In addition, when the Old file to be updated is on a remote share:

- SQLPathFinder will not copy the source file if it is locked and will abort the Smart Append job. Locking can occur if the file is opened in Excel in ReadWrite mode. SQLPathFinder will retry 3 times to copy a locked file with a wait time in between.
- SQLPathFinder will always use RoboCopy to first copy the remote file locally prior to the SmartAppend operation and will add a Date check to ensure that the copy completed successfully. SQLPathFinder will abort if the copy does not complete successfully including if the source and destination file dates do not match. This leverages a special behavior of RoboCopy.
- SQLPathFinder will always use RoboCopy to copy the final file back to the remote share after the Smart-Append operation and will again add a date check to ensure that copy was successful. If the copy was unsuccessful, SQLPathFinder will abort Smart-Append leaving the original old file untouched.
- When returning the final file to a remote share, SQLPathFinder will also copy the file back under a temporary name (i.e., "æ_temp" will be added to the file extension). Only if that copy was successful will SQLPathFinder delete the original file and rename the temporary file to its final file name.

These final steps should further harden the SmartAppend capability and counter network issues that have been known to occur from time to time on ScriptHost.

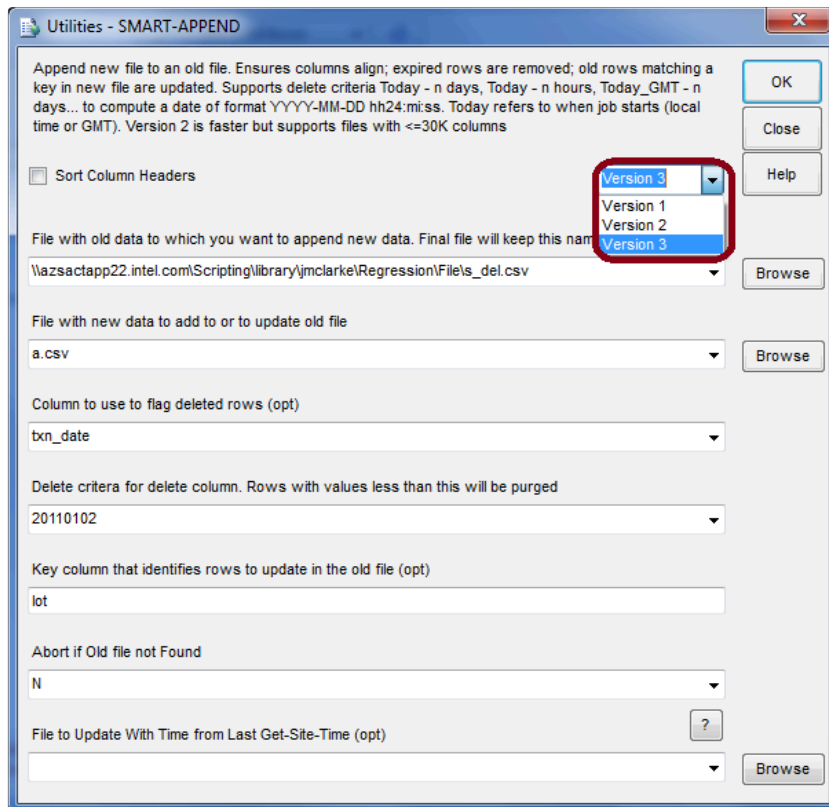


Figure SA-7 - Using smart-Append Version 3

[\[Return to Top\]](#)

Using the For-Loop and the Smart-Append Utility to run an incremental job

[\[Return to Top\]](#)

Sometimes you need to run queries incrementally to avoid exceeding database quota limits on a given database server as set by the database administrators. SQLPathFinder supports a **Run-Loop** utility which lets you run a query n-items at a time (e.g., 50 lots at a time). In addition, SQLPathFinder supports a **For-Loop** utility which, in combination with the **Smart-Append** utility, lets you run incremental date range extracts (e.g., to query data 5 days at a time) and that appends results into a single output file. In Figure SL-1 below, an Alarm history query is run that pulls data 1 day at a time for a two day period. The query contains the following components:

1. A **Delete** utility to remove the final output file (*AlarmData.tab*) before the query starts. This was done so you do not accidentally append data to an old version of this file.
2. A **For-Loop** utility which specifies a start, end, and step value
3. The **Query** with a date filter that leverages the Loop Counter and Step value configured in the For-Loop utility as well as other special tokens. The query writes output to file *AlarmHistory.tab*. This file will be overridden for each incremental run.
4. A **Smart-Append** utility that appends output data (and also provides other services such as updating rows in the old file based on rows in the new file, that deletes expired rows or ensures column headers are aligned in the case of a Pivot query where column headers may change from query run to query run.
5. An **End-Loop** utility which terminates the For-Loop utility.

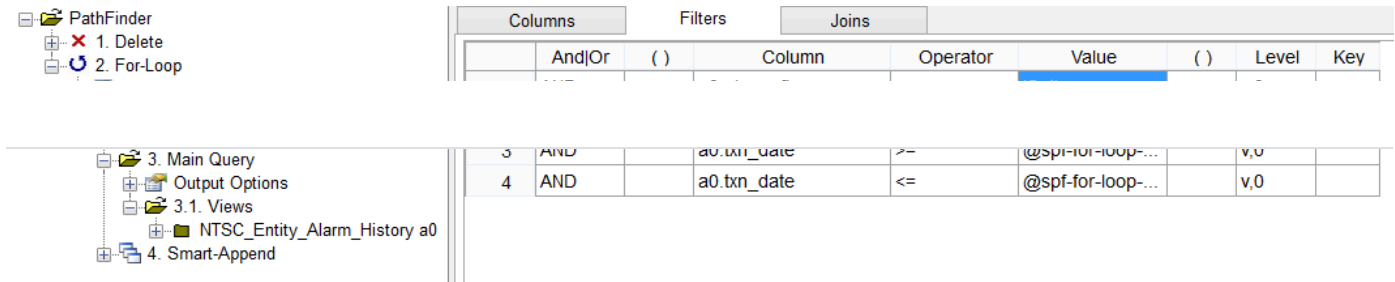


Figure FL-1 - Query that Pulls Data One Day at a Time

The For-Loop utility is configured as shown in *Figure FL-2*. The query is set up to pull data for the last 2 days in daily increments. The Start value is set to 0, the end value to 2, the Step value to 1 (for 1 day) and the loop suffix is defined as 1. The **loop suffix** ensures that For-Loop variables will be uniquely named in the case nested loops are used.

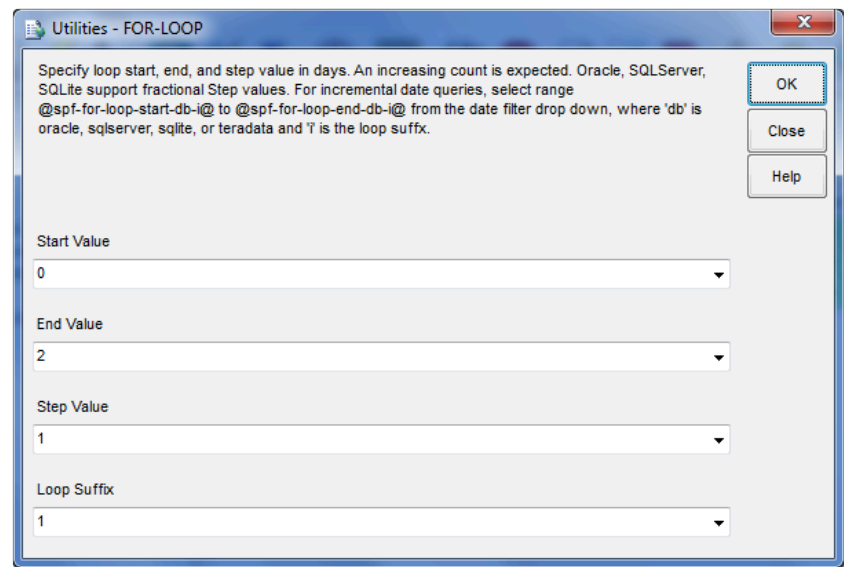


Figure FL-2 - The For-Loop Utility Form

The query will be a normal query with the exception that the date filter should leverage the special For-Loop variables. This is illustrated in *Figure FL-3*. The actual date filters are:

- 1. Incremental Start Date == **@spf-for-loop-start-oracle-1@**
- 2. Incremental End Date == **@spf-for-loop-end-oracle-1@**

The incremental date boundaries can be selected from the drop-down date list as suggested in *Figure FL-3*. Assuming the job execution start day is 12-Aug-2013 21:34:39 Az time (or 2013-08-13 04:34:39 GMT time) , date values for different iterations of the *For-Loop* will be translated to the following:

Loop Counter Value	Dates	Translated Local Time
2	CAST((FROM_TZ(CAST(TO_DATE('2013-08-13 04:34:39','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) - 2 CAST((FROM_TZ(CAST(TO_DATE('2013-08-13 04:34:39','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) + 1 - 2	2013-08-10 21:34:39 2013-08-11 21:34:39
1	CAST((FROM_TZ(CAST(TO_DATE('2013-08-13 04:34:39','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) - 1 CAST((FROM_TZ(CAST(TO_DATE('2013-08-13 04:34:39','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) + 1 - 1	2013-08-11 21:34:39 2013-08-12 21:34:39
0	CAST((FROM_TZ(CAST(TO_DATE('2013-08-13 04:34:39','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) - 0 CAST((FROM_TZ(CAST(TO_DATE('2013-08-13 04:34:39','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) + 1 - 0	2013-08-12 21:34:39 2013-08-13 21:34:39

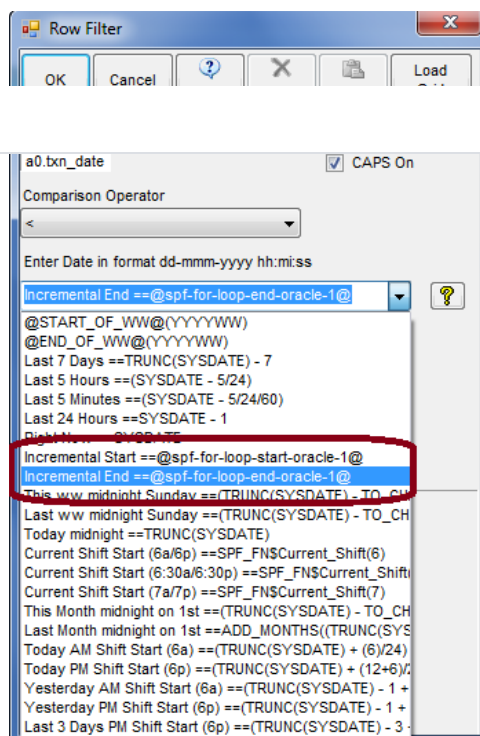


Figure FL-3 - Leveraging the For-Loop Variables in the Date Filter

Note that starting a For-Loop at 0 is a special case that will add one extra iteration to go beyond now to attempt to capture the most recently loaded data. If the start date is 1 or more, the job will end that number of days prior to the SQLPathFinder job start time.

The configuration for the *Smart-Append* utility is shown in Figure FL-4. Query output file *AlarmHistory.tab* from each incremental run is appended to file *AlarmData.tab*. In addition, column *txn_date* in the output is defined as a *Delete* column. Any rows in either the old or the new file that are older than **Today - 7 days** ago, where **Today** is the start day (at midnight) of the job as opposed to the date when this utility runs, will be purged. This feature ensures that there are no gaps in your incremental extract due to the length of time a query takes to run. For this scenario, no update columns are configured in the Smart-Append utility.

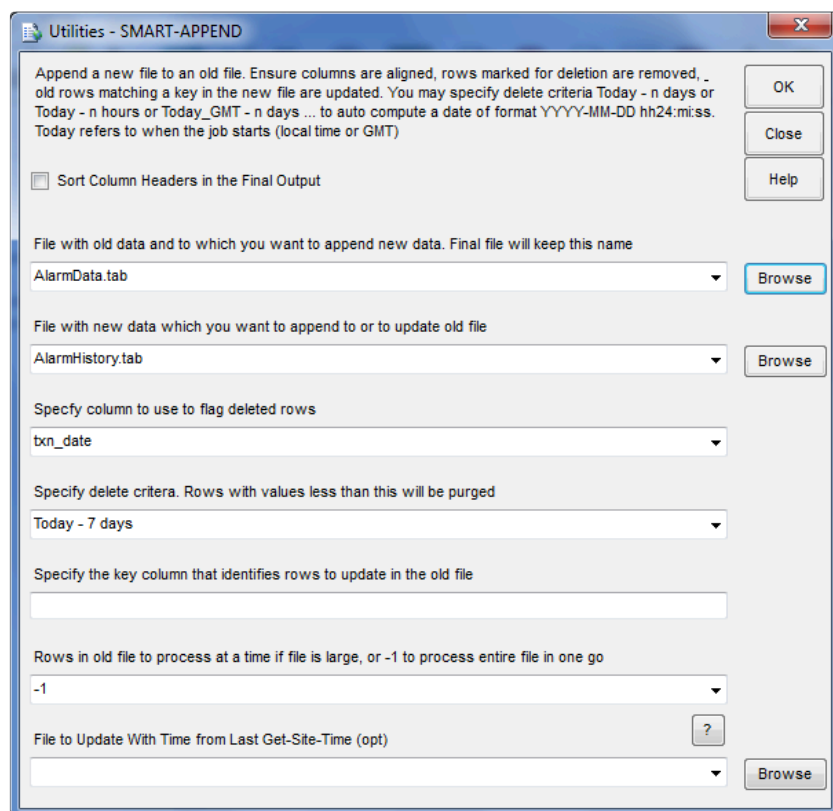


Figure FL-4 - Configuring the SMART-Append Utility

A sample log of the job is shown in Figure FL-5

```
Starting script v8.8 ... 08-Apr-2011 17:50:23
```

```
Starting Delete Applet, v2.2 ... 08-Apr-2011 17:50:23
```



```
Deleting "AlarmData.tab"  
Done ...
```

```
Loop Counter (spf-loop-ctr-1) = 0
```

```
Fetching MARS Data (F24.MAO) ... 08-Apr-2011 17:50:24  
Connected...
```

```
Done. 0 row(s) returned...  
08-Apr-2011 17:50:41
```

```
Starting Smart Append Applet, v1.1 .... 08-Apr-2011 17:50:41
```

```
*****  
Configuration options for Smart Append:  
Old File = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPathFinderIII\AlarmData.tab  
New File = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPathFinderIII\AlarmHistory.tab  
Delete Column = TXN_DATE  
Delete Criteria = < 2011-04-01 00:00:00 (case insensitive)  
Col to check for Updates =  
Sort Column Headers = N  
Does Source File Exist = N  
*****
```

```
New File Contains 0 row(s)...  
Deleting expired rows from New File... 08-Apr-2011 17:50:41  
Done. 0 row(s) deleted ... 08-Apr-2011 17:50:41  
Saving New File as Old File 08-Apr-2011 17:50:41
```

```
Done. 0 row(s) saved... 08-Apr-2011 17:50:41
```

```
Loop Counter (spf-loop-ctr-1) = 1
```

```
Fetching MARS Data (F24.MAO) ... 08-Apr-2011 17:50:42  
Connected...
```

```
Done. 10 row(s) returned...  
08-Apr-2011 17:50:52
```

```
Starting Smart Append Applet, v1.1 .... 08-Apr-2011 17:50:52
```

```
*****  
Configuration options for Smart Append:  
Old File = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPathFinderIII\AlarmData.tab  
New File = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPathFinderIII\AlarmHistory.tab  
Delete Column = TXN_DATE  
Delete Criteria = < 2011-04-01 00:00:00 (case insensitive)  
Col to check for Updates =  
Sort Column Headers = N  
Does Source File Exist = Y  
*****
```

```
Old File Contains 0 row(s)...  
New File Contains 10 row(s)...  
Deleting expired rows from New File... 08-Apr-2011 17:50:52  
Done. 0 row(s) deleted ... 08-Apr-2011 17:50:52  
Deleting expired rows from Old File... 08-Apr-2011 17:50:52  
Done. 0 row(s) deleted ... 08-Apr-2011 17:50:52  
Determine rows in old file to Update ... 08-Apr-2011 17:50:52  
Done ... 08-Apr-2011 17:50:52  
Determine unique column headers ... 08-Apr-2011 17:50:52  
Done ... 08-Apr-2011 17:50:52  
Arrange headers and Append data... 08-Apr-2011 17:50:52  
Processing Old File ... 08-Apr-2011 17:50:52  
Done ... 08-Apr-2011 17:50:52  
Processing New File ... 08-Apr-2011 17:50:52  
Done ... 08-Apr-2011 17:50:52  
Final File Contains 10 row(s)...  
Done ... 08-Apr-2011 17:50:52
```

```
Loop Counter (spf-loop-ctr-1) = 2
```

```
Fetching MARS Data (F24.MAO) ... 08-Apr-2011 17:50:53  
Connected...
```

```
Done. 8 row(s) returned...  
08-Apr-2011 17:51:02
```

```
Starting Smart Append Applet, v1.1 .... 08-Apr-2011 17:51:02
```

Configuration options for Smart Append:

Delete Criteria = < 2011-04-01 00:00:00 (case insensitive)

Col to check for Updates =

Sort Column Headers = N

Does Source File Exist = Y

Old File Contains 10 row(s)...

New File Contains 8 row(s)...

Deleting expired rows from New File... 08-Apr-2011 17:51:02

Done. 0 row(s) deleted ... 08-Apr-2011 17:51:02

Deleting expired rows from Old File... 08-Apr-2011 17:51:02

Done. 0 row(s) deleted ... 08-Apr-2011 17:51:02

Determine rows in old file to Update ... 08-Apr-2011 17:51:02

Done ... 08-Apr-2011 17:51:02

Determine unique column headers ... 08-Apr-2011 17:51:02

Done ... 08-Apr-2011 17:51:02

Arrange headers and Append data... 08-Apr-2011 17:51:02

Processing Old File ... 08-Apr-2011 17:51:02

Done ... 08-Apr-2011 17:51:02

Processing New File ... 08-Apr-2011 17:51:02

Done ... 08-Apr-2011 17:51:02

Final File Contains 18 row(s)...

Done ... 08-Apr-2011 17:51:02

For-Loop Completed ...

Figure FL-5 - Log of the Incremental Date Extract

ADDENDUM TO FOR LOOP:

For advance users, below are For-Loop variables you could reference between the start of the For-Loop and the associated End-Loop (assuming a suffix of 1):

Variable	Description
<<<spf-start-1>>>	This is the start value of the For Loop. Suffix is 1.
<<<spf-end-1>>>	This is the end value of the For Loop. Suffix is 1
<<<spf-step-1>>>	This is the step value of the For loop. Suffix is 1.
<<<spf-loop-ctr-1>>>	This is the For loop counter which is initially set to the start value but increments by the step value for each iteration of the loop. The loop ends when the counter exceeds the end value. Suffix is 1.
<<<spf-job-start-gmt-time>>>	This is a global variable available throughout SQLPathFinder that is set to the GMT date and time when the query job starts execution. You can use this as a reference point in a date filter. The format of the start day is YYYY-MM-DD 00:00:00 . E.g., 2011-04-08 00:00:00.

An alternative strategy for specifying an incremental date search with the FOR-Loop with Oracle is as follows.

1. Incremental Start Date == **Txn_Date** >= CAST((FROM_TZ(CAST(TO_DATE('<<<spf-job-start-gmt-time>>>','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) - <<<spf-loop-ctr-1>>>
2. Incremental End Date == **Txn_date** < CAST((FROM_TZ(CAST(TO_DATE('<<<spf-job-start-gmt-time>>>','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) + <<<spf-step-1>>> - <<<spf-loop-ctr-1>>>

This specification is what variables **@spf-for-loop-start-oracle-1@** and **@spf-for-loop-end-oracle-1@** translate to behind the scenes. Translation for different database types (e.g., SQL server, SQLite, Teradata...) are shown below:

Below are the incremental date range variables you can use for different database types, with "-i@" at the end of the variable name (e.g., -1@) referring to the loop suffix.

Oracle:

Incremental Start ==@spf-for-loop-start-oracle-1@

Incremental End ==@spf-for-loop-end-oracle-1@

SQL Server:

Incremental Start ==@spf-for-loop-start-sqlserver-1@

Incremental End ==@spf-for-loop-end-sqlserver-1@

SQLite:

Incremental Start ==@spf-for-loop-start-sqlite-1@

Incremental End ==@spf-for-loop-end-sqlite-1@

Teradata:

Incremental Start ==@spf-for-loop-start-teradata-1@

Incremental End ==@spf-for-loop-end-teradata-1@

These variables translate to the following macro strings. Macro Variable <<<spf-job-start-day>>> will translate to midnight on the day that the query is run. E.g., if the query was run sometime on July 11th, 2012, variable <<<spf-job-start-day>>> will translate to **2012-07-11 00:00:00**

ORACLE:

```
CAST((FROM TZ(CAST(TO DATE('<<<spf-job-start-gmt-time>>>','YYYY-MM-DD HH24:MI:SS') AS TIMESTAMP), 'GMT') AT LOCAL) AS DATE) - <<<spf-loop-
```

SQLITE:

```
DateTime('<<<spf-job-start-gmt-time>>>','Localtime','<<<spf-loop-ctr-1>>> Days')
DateTime('<<<spf-job-start-gmt-time>>>','Localtime','<<<spf-loop-ctr-1>>> Days','<<<spf-step-1>>> Days')
```

SQL SERVER:

```
CONVERT(CHAR(10),DATEADD(minute, DATEDIFF(minute, GETUTCDATE(), GETDATE()), '<<<spf-job-start-gmt-time>>>') - <<<spf-loop-ctr-1>>>, 23) + ' ' +
CONVERT(CHAR(8),DATEADD(minute, DATEDIFF(minute, GETUTCDATE(), GETDATE()), '<<<spf-job-start-gmt-time>>>') - <<<spf-loop-ctr-1>>>, 8)
CONVERT(CHAR(10),DATEADD(minute, DATEDIFF(minute, GETUTCDATE(), GETDATE()), '<<<spf-job-start-gmt-time>>>') + <<<spf-step-1>>> - <<<spf-loop-
ctr-1>>>, 23) + ' ' + CONVERT(CHAR(8),DATEADD(minute, DATEDIFF(minute, GETUTCDATE(), GETDATE()), '<<<spf-job-start-gmt-time>>>') + <<<spf-step-
1>>> - <<<spf-loop-ctr-1>>>, 8)
```

TERADATA:

```
CAST(CAST('<<<spf-job-start-gmt-time>>>+00:00' AS timestamp(0)) - INTERVAL '<<<spf-loop-ctr-1>>>' Day AS Char(25))
CAST(CAST('<<<spf-job-start-gmt-time>>>+00:00' AS timestamp(0)) - INTERVAL '<<<spf-loop-ctr-1>>>' Day + INTERVAL '<<<spf-step-1>>>' Day AS
CHAR(25))
```

[\[Return to Top\]](#)

Creating a Running "N" Day Extract File - Append New Data Since the Last Run, Delete Old Data, Update Changed Data

[\[Return to Top\]](#)
Introduction

With reduced quotas being implemented on the MARS/ARIES/OASys (MAO) database servers, it is important to write our queries efficiently to avoid exceed quota errors and long running jobs. One way to do this is by generating incremental extracts. That is, instead of extracting 60 days of data every day, build up a 60 data extract file and use SQLPathFinder's incremental capability to append new data since the last time the job ran, to update changed data, and to roll off data that is older than 60 days. Much of this can be achieved using SQLPathFinder's GET-SITE-TIME and SMART-APPEND utilities as illustrated in Figure Inc-1 below

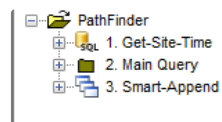


Figure Inc-1 - Utilities for an Incremental Extract

The **Get-Site-Time** utility obtains the current time at a site, the **Main Query** pulls incremental data since the last time the job ran, and the **Smart-Append** utility appends, updates, and deletes data in an old file based on criteria set and data extracted from the main query. If successfully run, the **Smart-Append** utility also updates a time file with the time retrieved by the last **Get-Site-Time** utility. That time file is also used in the Main Query of the next query run so that data pulled is greater than or equal to the last successful site extract Time. This process should greatly reduce the time taken to run your daily job.

The process is summarized in Figure Inc-2. In Step 5 below, the Old file is copied to the work folder if it is on a file share and processed together with the new file. Results are saved back to the Old file which is then re-copied to its original folder. If everything is successful, the SMART-Append utility will, if configured thus, update the last successful extract time to an Update file used by the Main Query (step 7 in Figure Inc-2)..

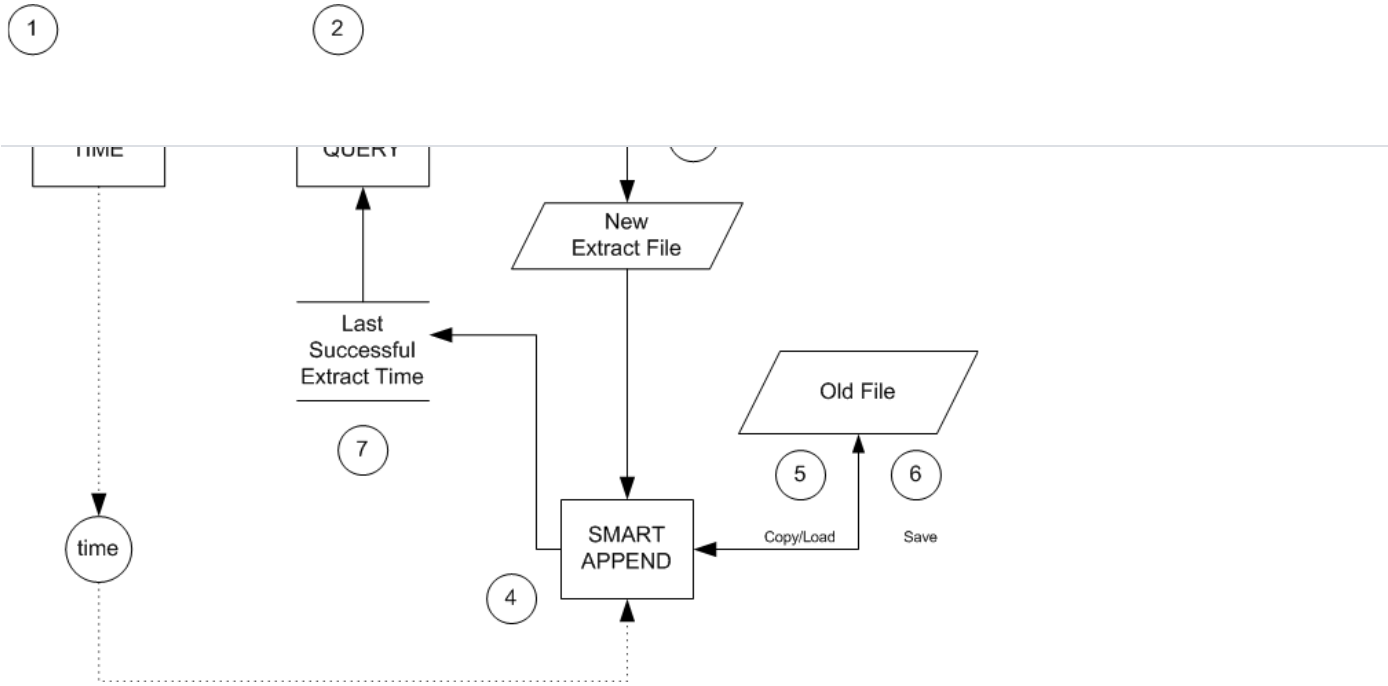


Figure Inc-2 - Incremental Query Flowchart

Sample SMART-APPEND Output

Sample Output of a SMART Append is illustrated below in the three tables. The assumption is that the delete criteria is set to remove rows where column "TXN_DATE" in the extract files is older than **Today - 30 days**, with today being assumed to be 9/24/2011. In addition, column "LOT" is assumed to be the key and any rows matching the lot column in the new file will update the old file. The SMART-APPEND will also integrate columns from the old and new file and any new columns will either be added to the end of the old file or stored in the file alphabetically if the SMART-APPEND is configured to order columns.

LOT	OPER	ENTITY	NEWCOL1	TXN_DATE
I1	o1	e1	aa1	2011-08-20 00:00:00
I2	o2	e2	aa2	2011-09-02 00:00:00
I3	o3	e3	aa3	2011-09-03 00:00:00
I4	o4	e4	aa4	2011-09-04 00:00:00
I5	o5	e5	aa5	2011-09-05 00:00:00

Table Inc-1 - Old File Before Processing

LOT	OPER	ENTITY	NEWCOL2	TXN_DATE
I0	O0	e0	bb0	2011-08-01 00:00:00
I2	o22	e22	bb2	2011-09-24 00:00:00
I6	o6	e6	bb6	2011-09-24 00:00:00

Table Inc-2 - New Extract File

LOT	OPER	ENTITY	NEWCOL1	TXN_DATE	NEWCOL2
I3	o3	e3	aa3	2011-09-03 00:00:00	
I4	o4	e4	aa4	2011-09-04 00:00:00	
I5	o5	e5	aa5	2011-09-05 00:00:00	
I2	o22	e22		2011-09-24 00:00:00	bb2
I6	o6	e6		2011-09-24 00:00:00	bb6

Table Inc-3 - Old File After Processing. Lot "I2" was updated and lot "I1" was deleted from the old file. Lot "I6" was added from the new file. Lot "I0" was removed from the new file as it was older than the delete criteria.

Configuring the Query

How to configure the incremental query illustrated in Figure Inc-1 is described in this section. A sample configuration for the **Get-Site-Time** utility is shown in Figure Inc-3. This utility is configured to retrieve the current time of the user's default MARS server. Recall that you must double-click utilities to configure them.

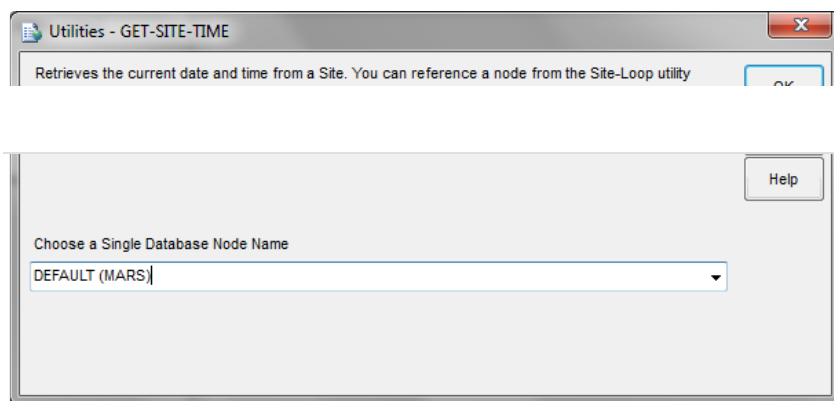


Figure Inc3 - Get-Site-Time utility. Configured to Get the Time using your Default MARS database

Figure Inc-4 shows the query design for a sample query. The main date filter is where column **lao_test_end_date** is greater than (**In Group(>=D)**) the seventh column in CSV file **LatestExtractTime.csv**. File **latestExtractTime.csv** stores the last date and time data was extracted from some prior extract, minus two hours. Table **LatestExtractTime.csv** is created and updated by either the **Smart-Append** or **Update-Time** utilities. These utilities update the time value obtained from the last run of the Get-Site-Time utility and also add several time offsets from that extract time. Filter **In Group (>=D)** will retrieve a character column of format YYYY-MM-DD hh24:mi:ss from a file and will convert it to a date time value required of the particular database. A greater than or equal to filter will be applied using the date column. The sample contents of file **LatestExtractTime.csv** are shown in Figure Inc-5.

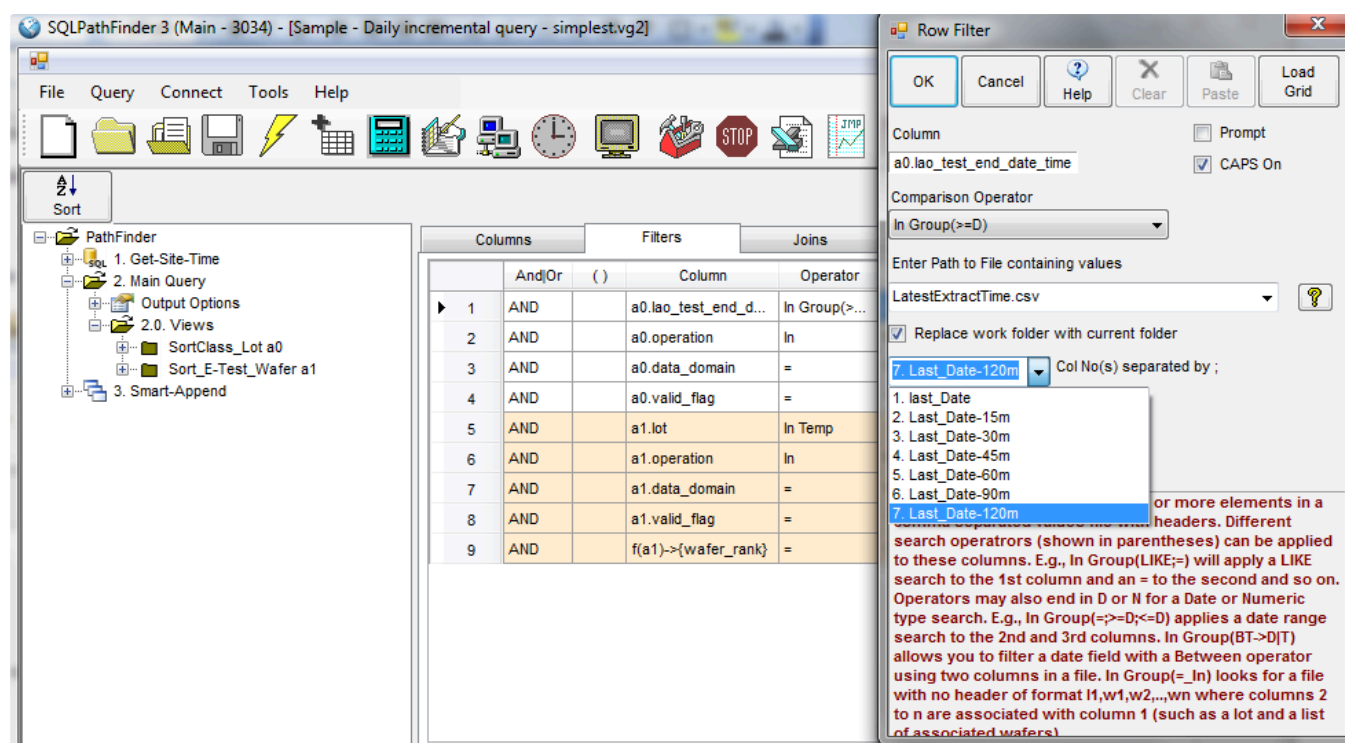


Figure Inc4 - The Main Query Which Searches Using a Date Column Greater than the last Successful Extract Time Minus 2 Hours

last_Date	Last_Date-15m	Last_Date-30m	Last_Date-45m	Last_Date-60m	Last_Date-90m	Last_Date-120m
2011-09-24 12:57:08	2011-09-24 12:42:08	2011-09-24 12:27:08	2011-09-24 12:12:08	2011-09-24 11:57:08	2011-09-24 11:27:08	2011-09-24 10:57:08

Figure Inc5 - Sample Contents of Time File LatestExtractTime.csv. Column Last_Date Stores the last successful extract time. Additional columns store offsets in minutes from that Extract Time.

Figure Inc-6 shows a SMART-APPEND configuration. Table **New.Tab** is set to be integrated into table **Old.Tab** located on a file share. Data from the two files are to be removed if column **Txn_Date** is older than Today's date minus 30 days. In addition column **LOT** is identified as the key and any data in the new file that has the same key value as that in the old file will override that row in the Old file. The row count to process in the two files is also set to -1 implying that all rows will be processed in one go. By setting a row count, data will be processed that many rows at a time in case of very large files. If the SMART-APPEND is successful, the last Get-Site-Time value retrieved will be added to file **LatestExtractTime.csv** located in the user's work folder.

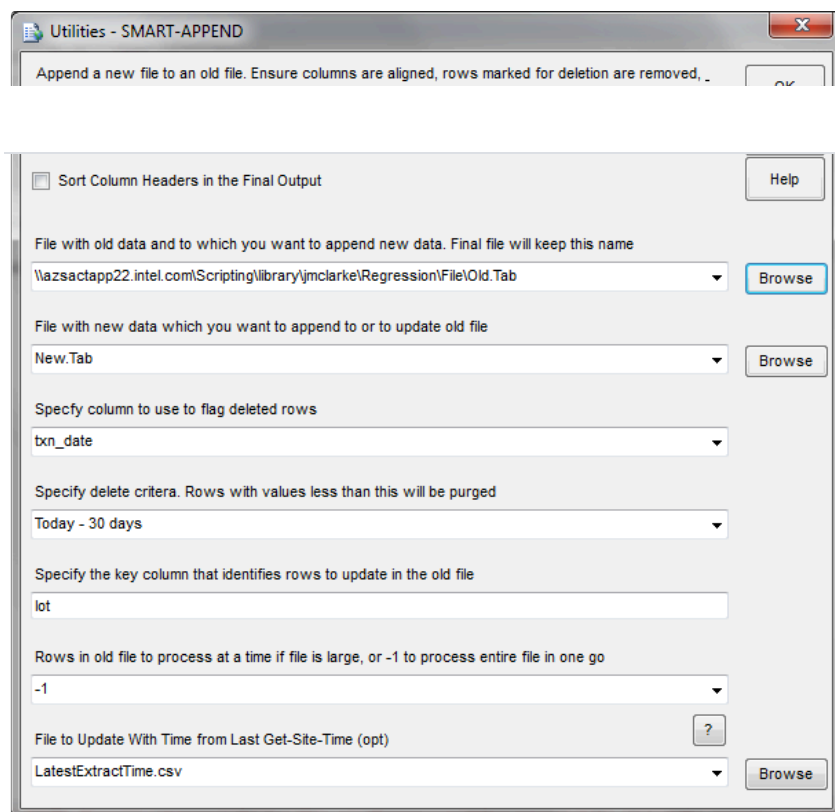


Figure Inc-6 - SMART Append Configured to Integrate Data from New.Tab into Old.Tab.

A sample log of a Job with a Get-Site-Time and a Smart-APPEND utility (minus the Main Query) may be as follows..

Get Time at F12.MAO

Getting Data from F12.MAO ... 24-Sep-2011 12:56:57
Connected...

Done. 1 row(s) returned...

Starting Smart Append Applet, v3.0 24-Sep-2011 12:57:10

```
*****
Smart Append options:
Old File = \\azsactapp22.intel.com\scripting\library\jmclarke\Regression\File\Old.tab
New File = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPathFinderIII\New.tab
Delete Column = TXN_DATE
Delete Criteria = < 2011-08-25 00:00:00 (case insensitive)
Update Column = LOT
Sort Headers = N
Source File Exists = Y
Process Row Count = -1
Copy Old File local = Y (11628_Old.tab)
Update Site Time = Y (LatestExtractTime.csv)
*****
```

```
New File Contains 3 row(s)...
Deleting expired rows from New File... 24-Sep-2011 12:57:16
Done. 1 row(s) deleted ... 24-Sep-2011 12:57:16
Determine unique column headers ... 24-Sep-2011 12:57:16
Done ... 24-Sep-2011 12:57:16
```

```
Processing Old File ... 24-Sep-2011 12:57:16
Deleting expired rows... 24-Sep-2011 12:57:16
Done. 1 row(s) deleted ... 24-Sep-2011 12:57:16
Determine rows to update... 24-Sep-2011 12:57:16
Done ... 24-Sep-2011 12:57:16
Remove rows to be updated... 24-Sep-2011 12:57:16
Done. 1 row(s) deleted... 24-Sep-2011 12:57:16
Arrange headers and Append data... 24-Sep-2011 12:57:16
Done ... 24-Sep-2011 12:57:17
```

```
Processing New File ... 24-Sep-2011 12:57:17
Arrange headers and Append data... 24-Sep-2011 12:57:17
Done ... 24-Sep-2011 12:57:18
```

```
Total Rows in Old File : 5
Total Rows removed from Old File: 2
Total Rows in new File : 3
Total Rows removed from new File: 1
```

Total Rows in Final File : 5

Done ... 24-Sep-2011 12:57:20

Figure Inc-6 - Sample Log of a Get-Site-Time utility and a Smart-Append Utility

Creating the Initial Large Extract File

One initial challenge is to initially create a large extract file under reduced quotas. In SQLPathFinder this can be done using the item count on the *In Temp* filter operator. That is you break your query into a two view query. The first query simply pulls lots (or items such as work weeks...) processed in the time period, which usually finishes within the quota time period. The second query processes those lots or items so many items at a time. In addition, for the first query run, you would not need to use the extract time file but can simply specify a date range. This is illustrated in Figure Inc-7. For item (1) in Figure Inc-7, see how the date range is initially set to pull data since August 1st, 2011 from view SortClass_Lot. This simply extracts all Sort lots processed since that time. In the second query (Sort_E-Test_Wafer), an In Temp filter is used (2) to read in that lot list 10 lots at a time (3). The two queries would also need to be joined in this case by at least the lot number. Note that view WIP_Lot_History may also be a good view to use to retrieve an initial lot list that has completed processing at one or more operations.

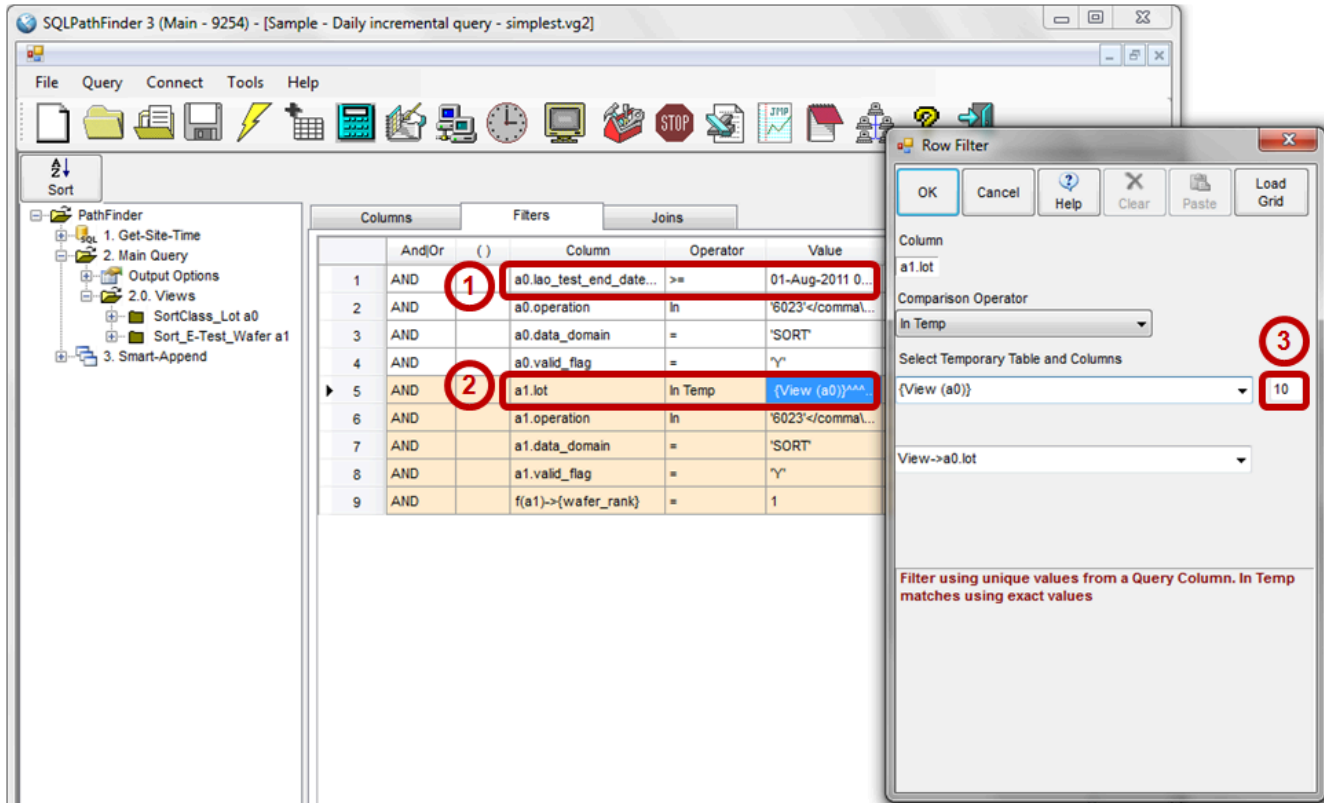


Figure Inc-7 - Ensuring the Initial Query runs to Completion Using an Item Count on the *In Temp* filter.

[\[Return to Top\]](#)

The Write-File Utility

[\[Return to Top\]](#)

The **Write-File** utility simply lets you add any text you wish written to a file into the SQLPathFinder notepad like editor and to save it to a particular file. This could be a vbscript program you wish to run later or a macro file you wish to use later in the SQLPathFinder job and so on.

Note that the file to save the text to is configured in the Write-File editor as illustrated below in Figure WF-1. Select menu option **Output File -> Output File** in the editor and enter a file name or browse to a file.

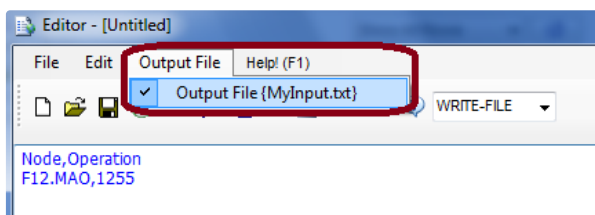


Figure WF-1 - Assigning a File to write Text to in the write-File Editor

[\[Return to Top\]](#)

[\[Return to Top\]](#)

The **Site-Loop** utility allows you to efficiently loop through a list of database nodes. Site-Loop also allows a job to continue even if one of the site-queries fails. The affected loop iteration would end on the query failure, and the job will resume at the beginning of the next Site-Loop iteration. *Any Update-Time actions within the Site-Loop (whether part of the Smart-Append or Update-Time utility) will not happen, so that the next time the job runs, it will try again to pull the data that it failed to pull this time, and will continue to do so until that site is finally successful.* Below is an example of the new behavior.

A sample query is shown in Figure SL-1. The query deletes an output file, uses Site-Loop to loop through a join query using a list of nodes and, as part of the Site-Loop, executes a pre/post query, runs a few utilities, and leverages a Smart-Append utility to union the query results into a single output file. The Site-Loop node list is shown in Figure SL-2. Note that the second node (**ABC.MARS**) is invalid to simulate a scenario where the second node failed the connect or failed during query execution. A sample log of the results is shown in Figure SL-3. Note that the job errors on node 2 (log file section in red) and immediately jumps to the next Site Loop iteration, skipping all steps between the query that failed and the end of the utility. At this time, Site-Loop will only behave this way with failed intermediate queries and not with utilities as we did not want to risk potential corruption of any data set due to unintended consequences. A severe failure with a utility (e.g., Smart-Append) will still result in the entire job aborting.

FIGURE SL-1: Site Loop Query

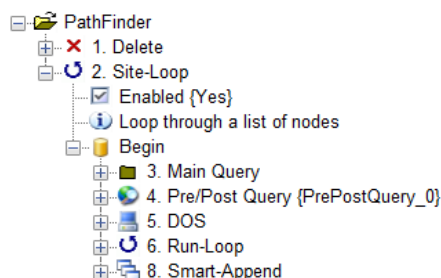


FIGURE SL-2: SITE Loop Definition

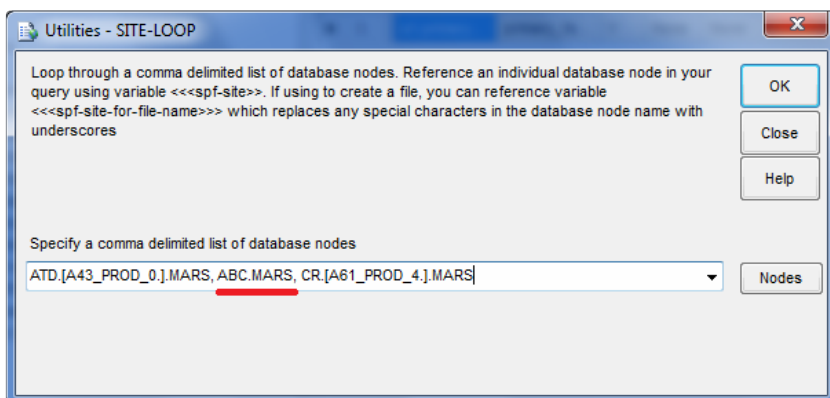


FIGURE SL-3: Log of the Site Loop Job

```
Starting Delete Applet, v2.3 ... 17-Dec-2012 15:10:11 (DELETE UTILITY)
Deleting "Test.Tab"
Done ...

Start Site Loop Utility ... (SITE-LOOP UTILITY)
===== (LOOP 1)
Site = ATD.[A43_PROD_0].MARS
=====

Fetching MARS Data (ATD.[A43_PROD_0].MARS) ... 17-Dec-2012 15:10:11
Connected...
Done. 1 row(s) returned...
17-Dec-2012 15:10:12

Fetching MARS Data (ATD.[A43_PROD_0].MARS) ... 17-Dec-2012 15:10:12
Connected...
Done. 1 row(s) returned...
17-Dec-2012 15:10:12

Getting Data Using SQLite ... 17-Dec-2012 15:10:12
Starting Data Import and SQL Query ...
Done... 17-Dec-2012 15:10:12

Fetching Text (SQLite) Data ... 17-Dec-2012 15:10:12
Starting Data Import and SQL Query ...
```


Done... 17-Dec-2012 15:10:12

Done. 1 row(s) returned... 17-Dec-2012 15:10:12

C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III

(LOOP 1 - DOS UTILITY)

Start Looping Utility ...
LOOP UTILITY)

(LOOP 1 - RUN-

Looping through 1 row(s) in C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\Test1.tab in batches of 1
 Processing Rows 1 to 1

Loop Completed ...

Starting Smart Append Applet, v3.4 17-Dec-2012 15:10:13

(LOOP 1 - SMART-APPEND)

Smart Append options:

```

Old File           = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\Test.Tab
New File           = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\Test1.tab
Delete Column      =
Delete Criteria    = None
Update Column      =
Sort Headers       = Y
Source File Exists = N
Process Row Count  = -1
Copy Old File local = N
Update Site Time   = N
  
```

```

New File Contains 1 row(s)...
No row deletion required...
Determine unique column headers ... 17-Dec-2012 15:10:13
  Done ... 17-Dec-2012 15:10:13
Processing New File ... 17-Dec-2012 15:10:13
  Arrange headers and Append data... 17-Dec-2012 15:10:13
    Done ... 17-Dec-2012 15:10:13

Total Rows in Final File      : 1

Done ... 17-Dec-2012 15:10:13
  
```

(LOOP 2)

Site = ABC.MARS

Fetching MARS Data (ABC.MARS) ... 17-Dec-2012 15:10:13

ERROR:

ORA-12154: TNS:could not resolve the connect identifier specified

SP2-0751: Unable to connect to Oracle. Exiting SQL*Plus

There was a SQL error in your last query. Code=1

 Query Error occurred in last Site Loop. Going on to next Loop iteration..

(LOOP 3)

Site = CR.[A61_PROD_4.].MARS

Fetching MARS Data (CR.[A61_PROD_4.].MARS) ... 17-Dec-2012 15:10:23

Connected...

Done. 1 row(s) returned...
 17-Dec-2012 15:10:27

Fetching MARS Data (CR.[A61_PROD_4.].MARS) ... 17-Dec-2012 15:10:27

Connected...

Done. 1 row(s) returned...
 17-Dec-2012 15:10:32

Getting Data Using SQLite ... 17-Dec-2012 15:10:32

Starting Data Import and SQL Query ...

Done... 17-Dec-2012 15:10:3

Fetching Text (SQLite) Data ... 17-Dec-2012 15:10:32

```
PRIMARY_FACILITY PRIMARY_FACILITY_1 MARSSCHEMANAME MARSSCHEMANAME_11 (LOOP 3 - PRE/POST QUERY)
-----
A06 A06 A61_PROD_4 A61_PROD_4
Done. 1 row(s) returned... 17-Dec-2012 15:10:32
```

C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III (LOOP 3 - DOS UTILITY)

Start Looping Utility ... (LOOP 3 - RUN-
LOOP UTILITY)
 Looping through 1 row(s) in C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\Test1.tab in batches of 1
 Processing Rows 1 to 1
 Loop Completed ...

Starting Smart Append Applet, v3.4 17-Dec-2012 15:10:32 (LOOP 3 - SMART-APPEND)

```
=====
Smart Append options:
  Old File      = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\Test.Tab
  New File      = C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\Test1.tab
  Delete Column =
  Delete Criteria = None
  Update Column =
  Sort Headers  = Y
  Source File Exists = Y
  Process Row Count = -1
  Copy Old File local = N
  Update Site Time = N
=====
```

```
New File Contains 1 row(s)...
No row deletion required...

Determine unique column headers ... 17-Dec-2012 15:10:32
Done ... 17-Dec-2012 15:10:32

Processing Old File ... 17-Dec-2012 15:10:32
  No row deletion required...
  No update required...
  Arrange headers and Append data... 17-Dec-2012 15:10:32
  Done ... 17-Dec-2012 15:10:32

Processing New File ... 17-Dec-2012 15:10:32
  Arrange headers and Append data... 17-Dec-2012 15:10:32
  Done ... 17-Dec-2012 15:10:32

Total Rows in Old File      : 1
Total Rows removed from Old File: 0
Total Rows in New File      : 1
Total Rows removed from New File: 0
Total Rows in Final File    : 2

Done ... 17-Dec-2012 15:10:32
```

```
=====
Site-Loop Completed ...
=====
```

Setting a Temporary Site with the Site-Loop Utility When Debugging a Query

For debugging individual query steps within a Site-Loop, use the Temporary SPF Site option, shown in Figure SL-4. Note the "D1C.MAO" that has been entered into the text box. (After you have entered the desired node name to use for testing, just click anywhere outside that box to complete the entry.) Then you can right-click -> Run any step that contains a <<spf-site>> macro as the node name. To remove the temporary node, simply erase it and again click anywhere outside of the text box.

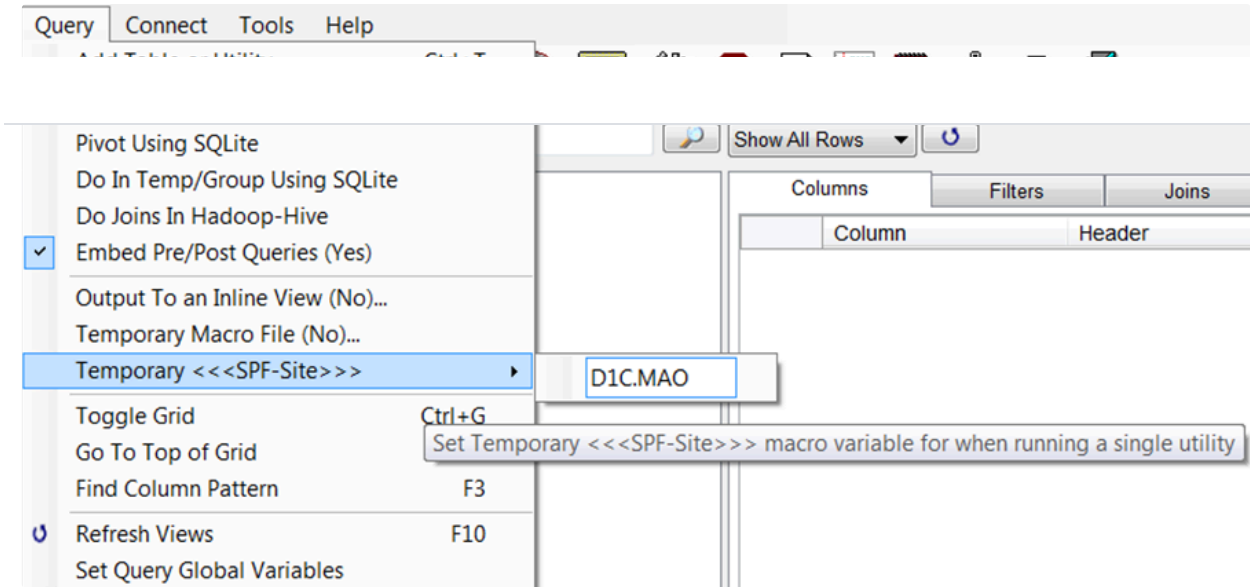


FIGURE SL-4 - Setting a Temporary Site when Debugging the Site-Loop Utility

[\[Return to Top\]](#)

Notes on the Run-Loop Utility Used with Macros

[\[Return to Top\]](#)

The Run-Loop utility can optionally let jobs continue even if there is a query error during a loop iteration. This can be especially important when used in conjunction with the Macro utility. For cases where Run-Loop is configured to continue on despite query failures, the affected loop iteration would end on the query failure, and the job will resume at the beginning of the next Run-Loop iteration. Any Update-Time actions within the Run-Loop/Macro (whether part of the Smart-Append or Update-Time utility) will not happen, so that the next time the job runs, it will try again to pull the data that it failed to pull this time, and will continue to do so until the affected query is finally successful. Below is an example of the new behavior.

A sample query is shown in Figure RL-1. The query uses Run-Loop to loop through a macro file (MacroTest.csv) created by a write-File utility, and passes data to the macro utility one row at a time for execution. As part of the Run-Loop/Macro, there is a join query. The contents of the macro file created by the Write-File utility is shown in Figure RL-2, and the definition of the Run-Loop utility is shown in Figure RL-3. Note that the second node (ABC.MAO) is invalid to simulate a scenario where the second node failed the connect or failed during query execution. A sample log of the results is shown in Figure SL-4. Note that the job errors on node 2 (log file section in red) and immediately jumps to the next Run Loop iteration, skipping all steps between the query that failed and the associated End-Loop. Setting option **For Query errors, go to the Next Loop Iteration** in Run-Loop utility to N, will cause the query to abort after any query failure. At this time, Run-Loop will only behave this way with failed intermediate queries and not with utilities. A severe failure with a utility (e.g., Smart-Append) will still result in the entire job aborting.

Figure RL-1: Run-Loop/Macro Query

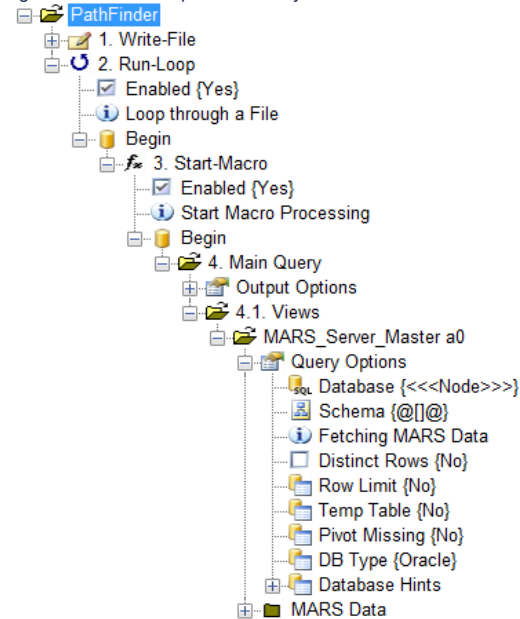


Figure RL2: Contents of Macro File. Data is Written to MacroTest.csv

Node

F12.MAO
ABC.MAO
F32.MAO

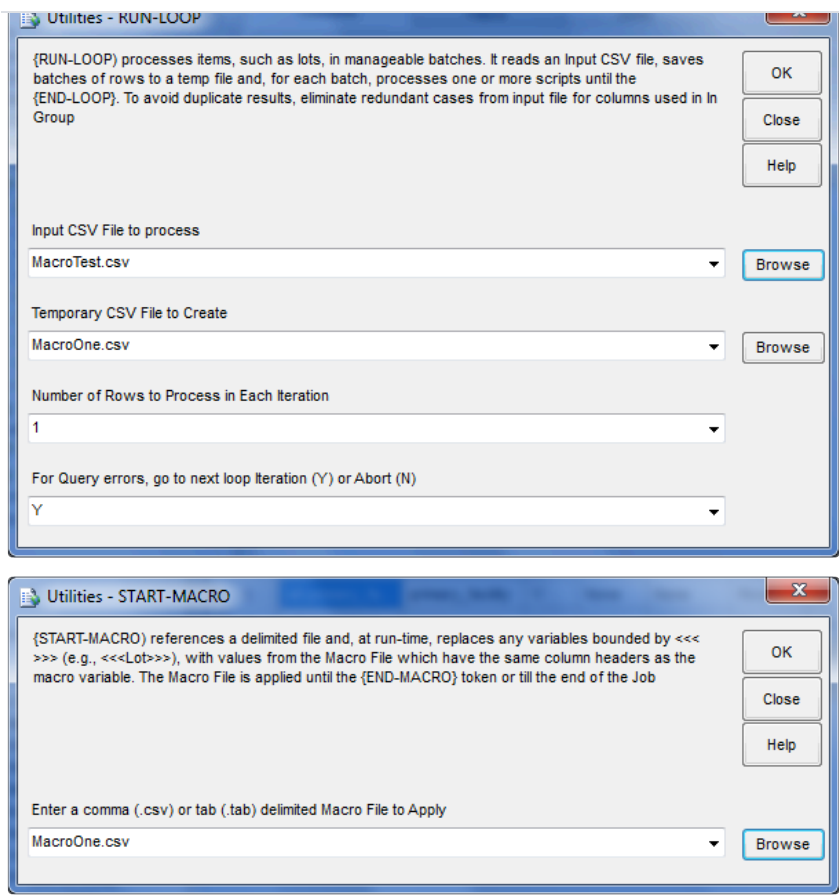


Figure RL4: Log of the Execution of the Query. Query Failure is Shown in Red

```
Starting Write-FILE Applet, v1.0 .... 21-Dec-2012 13:31:06
Saving data to: C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\MacroTest.csv
Done ...

Start Looping Utility ...
Looping through 3 row(s) in C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\MacroTest.csv in batches
of 1
=====
Processing Rows 1 to 1
===== - LOOP - 1

Start Macro processing: C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\MacroOne.csv

Fetching MARS Data (F12.MAO) ... 21-Dec-2012 13:31:07
Connected...

Done. 1 row(s) returned...
21-Dec-2012 13:31:30

Fetching MARS Data (F12.MAO) ... 21-Dec-2012 13:31:30
Connected...

Done. 1 row(s) returned...
21-Dec-2012 13:31:32

Getting Data Using SQLite ... 21-Dec-2012 13:31:32

Starting Data Import and SQL Query ...
Done... 21-Dec-2012 13:31:32

primary_facility      marsservername      primary_facility_1      marsschemaname
-----
F12                   F12PDB110N5        F12                     F22_PROD_1

Done. 1 row(s) returned... 21-Dec-2012 13:31:32
```

Macro Processing Completed ...

Start Macro processing: C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\MacroOne.csv

Fetching MARS Data (ABC.MAO) ... 21-Dec-2012 13:31:32

ERROR:

ORA-12154: TNS:could not resolve the connect identifier specified

SP2-0751: Unable to connect to Oracle. Exiting SQL*Plus

There was a SQL error in your last query. Code=1

Macro Processing Completed ...

Query Error occurred in last Loop. Going on to next Loop iteration..

===== - LOOP 3

Processing Rows 3 to 3

=====

Start Macro processing: C:\Users\jmclarke\Documents\SQLPathFinder_WorkFolder\SQLPath Finder III\MacroOne.csv

Fetching MARS Data (F32.MAO) ... 21-Dec-2012 13:31:44

Connected...

Done. 1 row(s) returned...

21-Dec-2012 13:31:45

Fetching MARS Data (F32.MAO) ... 21-Dec-2012 13:31:46

Connected...

Done. 1 row(s) returned...

21-Dec-2012 13:31:47

Getting Data Using SQLite ... 21-Dec-2012 13:31:47

Starting Data Import and SQL Query ...

Done... 21-Dec-2012 13:31:47

primary_facility	marsservername	primary_facility_1	marsschemaname
-----	-----	-----	-----
F32	F32PDB110N5	F32	F32_PROD_1

Done. 1 row(s) returned... 21-Dec-2012 13:31:47

Macro Processing Completed ...

=====

Loop Completed ...

=====

[\[Return to Top\]](#)

Date-Of-File Utility

[\[Return to Top\]](#)

The Date-of-File utility returns the last modified date of a file to a local environment variable. This utility in conjunction with the If-Then utility can be used to detect if a file has the same modified end date and time. This could, for example, be used to detect whether the source and destination files are identical following a RoboCopy as the RoboCopy utility only sets the modified date of the destination file to the source file date after the RoboCopy has been completed successfully.

[\[Return to Top\]](#)

Linux-Windows Cross-Over Share:

The Linux application server accessible from Python or R (by checking the **Run on Linux** checkbox as illustrated in Figure 2 above) supports a file share that is accessible from both MicroSoft Windows as well as Linux. Hence you can place files to run on Linux on that share from your windows client and access them from Linux, or write output from Linux there and read that output from SQLPathfinder. Two SQLPathFinder variables are provided to assist you with accessing that share

SQLPathFinder Variable	File Share Path	Example
<<<spf-linux-crossover-share-with-slash>>>	/hadoop/staging/user/<user-name>/	/hadoop/staging/user/jmclarke/
<<<spf-win-crossover-share-with-slash>>>	\\atdpfile1.ch.intel.com\hadoop_staging\user\<user-name>\	\\atdpfile1.ch.intel.com\hadoop_staging\user\jmclarke\

[\[Return to Top\]](#)

Get-TCA Utility

[\[Return to Top\]](#)

The **Get-TCA** utility lets you run canned TCA reports from SQLPathFinder. The utility accepts two arguments:

- an XML input file specifying the report to run as well as the selection criteria and output columns (where required), as well as
- the name of the CSV or TAB output file to create. CSV files should have an extension of .csv and tab files an extension of .tab.

Please ensure you have access to [CATTS UI](#) before trying out below reports from SQLPathFinder. Note too that the TCA web service utility writes the intermediate file output either to folder:

- \\atdpfile3.ch.intel.com\catts\export\csv\sqlpf\ or to
- \\atdpfile3.ch.intel.com\catts\export\csv\sqlpf\Foundry

...depending on whether this is a normal or a Foundry query. SQLPathFinder then copies the file to your desired destination folder. Do make sure you transfer the file to an appropriately secured file share.

If you run into errors using the TCA web service, you may reference the [SQLPathFinder TCAWebservice Trouble shooting guide](#)

To add the **Get-TCA** utility to your query, choose menu option **Query -> Add Table or Utility** from the Query design form. Then select Data Area -> **Utilities** on the **Add a View or Utility** window, and double-click the **Get-TCA** object. This is illustrated in Figure GT1 below. You can then position the utility in your query using drag-and-drop. To configure the utility, double-click it. This invokes the **Get-TCA** utility form where you specify both the XML input file and the output file as described above. Table GT-3 below describes the format of the input specification file.

In addition, a sample SQLPathFinder query that generates different permutations of XML files and runs all available reports can be obtained [here](#).

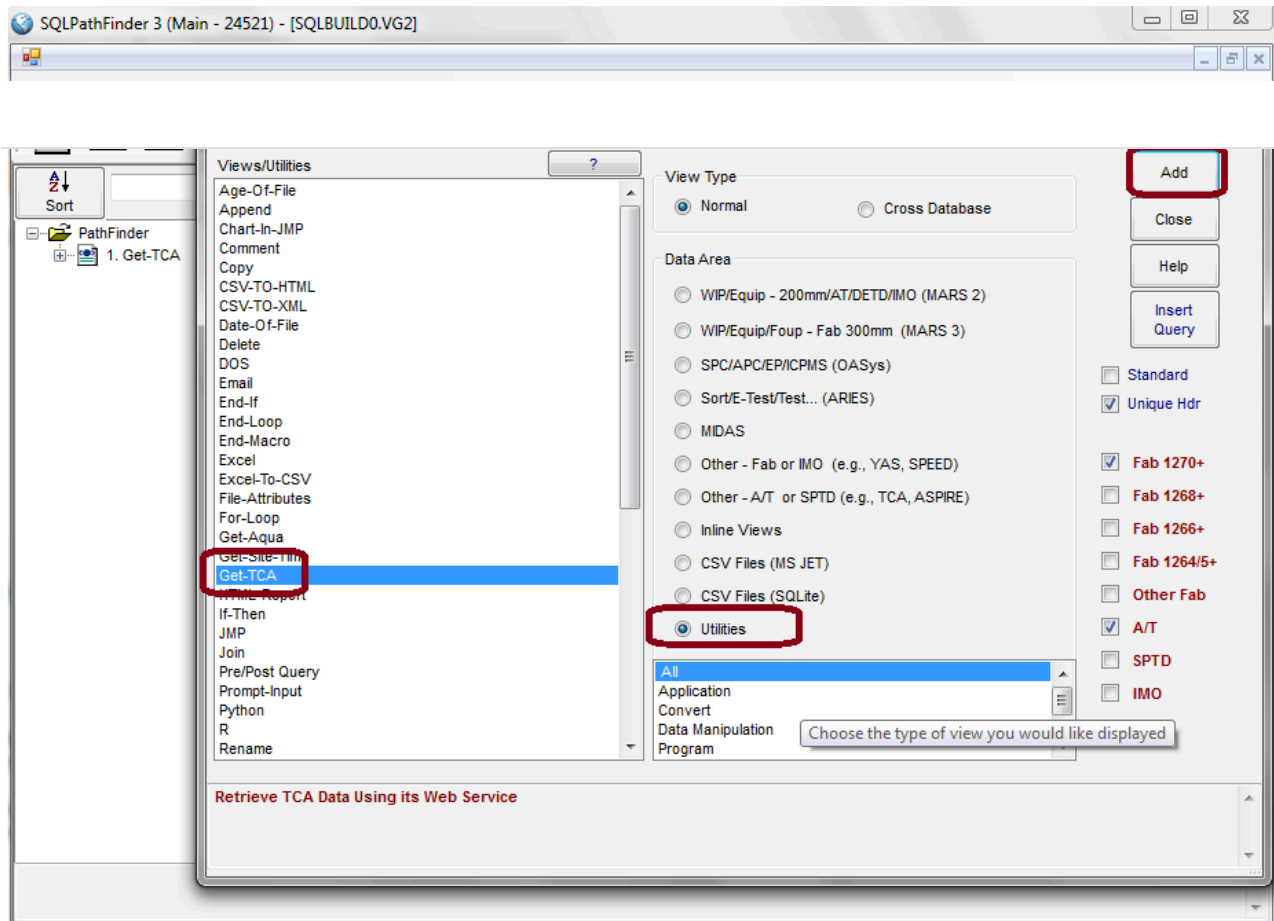


Figure GT1 - Adding the **Get-TCA** Utility to your query.

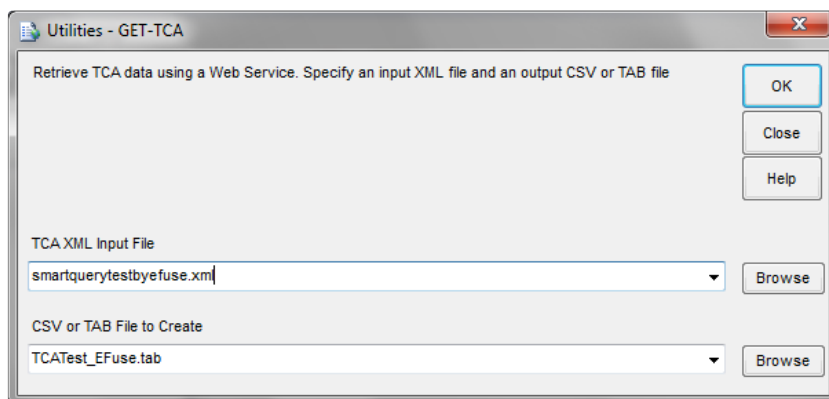


Figure GT2 - Configuring the **Get-TCA** Utility

Details on input XML format for TCAWebService operations.

TCASampleProcessingReport			
TCASampleProcessingReport	Input XML samples	Comments	
	<i>Required XML elements</i> <i>Output column elements where appropriate : set value = True to have in output</i>		
DiePrep Wafer Processing Sequence Report	<pre> <CattsTcaServiceRequest> <CATTSOperationName>AdvancedDBSearchRequest</CATTSOperationName> <ReportName>DP Wafer sequence report</ReportName> <InputData> <InputDataItem type="Die prep lot">D2073520</InputDataItem> <InputDataItem type="Die prep lot">H1316855</InputDataItem> <InputDataItem type="operation">1107</InputDataItem> <InputDataItem type="operation">1101</InputDataItem> <InputDataItem type="operation">1105</InputDataItem> </InputData> </CattsTcaServiceRequest> </pre>	• Repeat "InputDataItem" "Die prep lot" for additional lot numbers in input • Repeat "InputDataItem" "operation" for additional operations in input • If there is invalid lot(s) in the input, report generation will fail with error message. Remove the invalid lot(s) from input and re-run the report	

Integration Report : Lot (DATASOURCE = ULT)	<pre> <CattsTcaServiceRequest> <CATTSOperationName>SqlpfLotToUnitLevelIntegrationRequest</CATTSOperationName> <InputType>LOT</InputType> <InputData> <InputDataItem type="OPERATION">1210</InputDataItem> <InputDataItem type="EFUSE">O126738M,629,-3,-11</InputDataItem> <InputDataItem type="DATASOURCE">ULT</InputDataItem> // ULT Specific OutPutColumns <InputDataItem type="UPI">True</InputDataItem> <InputDataItem type="StripId">True</InputDataItem> <InputDataItem type="StripX">True</InputDataItem> <InputDataItem type="StripY">True</InputDataItem> <InputDataItem type="BabyMarkId">True</InputDataItem> <InputDataItem type="UnitStatusNumber">True</InputDataItem> <InputDataItem type="UnitProcessDateTime"> True </InputDataItem> <InputDataItem type="Bin">True</InputDataItem> <InputDataItem type="ErrorCode">True </InputDataItem> </InputData> </CattsTcaServiceRequest> </pre>	Set Output Column value = False if you dont need specific column(s) in output	
Integration Report : Efuse (DATASOURCE = ULT)	<pre> <CattsTcaServiceRequest> <CATTSOperationName>SqlpfLotToUnitLevelIntegrationRequest</CATTSOperationName> <InputType>EFUSE</InputType> <InputData> <InputDataItem type="OPERATION">1210</InputDataItem> <InputDataItem type="EFUSE">O126738M,629,-3,-11</InputDataItem> <InputDataItem type="EFUSE">F2106760,134,0,2</InputDataItem> <InputDataItem type="DATASOURCE">ULT</InputDataItem> // ULT Specific OutPutColumns <InputDataItem type="UPI">True</InputDataItem> <InputDataItem type="StripId">True</InputDataItem> <InputDataItem type="StripX">True</InputDataItem> <InputDataItem type="StripY">True</InputDataItem> <InputDataItem type="BabyMarkId">True</InputDataItem> <InputDataItem type="UnitStatusNumber">True</InputDataItem> <InputDataItem type="UnitProcessDateTime"> True </InputDataItem> <InputDataItem type="Bin">True</InputDataItem> <InputDataItem type="ErrorCode">True </InputDataItem> </InputData> </CattsTcaServiceRequest> </pre>	- If specified Efuse(s) are not present an invalid unit file is generated - Set Output column value = False if you dont need specific column(s) in output	
Integration Report : Efuse (DATASOURCE = Aries)	<pre> <CattsTcaServiceRequest> <CATTSOperationName>SqlpfLotToUnitLevelIntegrationRequest</CATTSOperationName> <InputType>LOT</InputType> <InputData> <InputDataItem type="OPERATION">6881</InputDataItem> <InputDataItem type="OPERATION">2172</InputDataItem> <InputDataItem type="OPERATION">7571</InputDataItem> <InputDataItem type="OPERATION">7310</InputDataItem> <InputDataItem type="OPERATION">7297</InputDataItem> <InputDataItem type="LOT">7409D059</InputDataItem> <InputDataItem type="LOT">8415C957</InputDataItem> <InputDataItem type="LOT">D407A758</InputDataItem> <InputDataItem type="DATASOURCE">ARIES</InputDataItem> // ARIES Specific OutPutColumns <InputDataItem type="UPI">True</InputDataItem> <InputDataItem type="StripId">True</InputDataItem> <InputDataItem type="StripX">True</InputDataItem> <InputDataItem type="StripY">True</InputDataItem> <InputDataItem type="BabyMarkId">True</InputDataItem> <InputDataItem type="UnitStatusNumber">True</InputDataItem> <InputDataItem type="UnitProcessDateTime">True</InputDataItem> <InputDataItem type="SummaryNumber">True</InputDataItem> <InputDataItem type="SummaryLetter">True</InputDataItem> <InputDataItem type="WithinSessionSequenceNumber">True</InputDataItem> <InputDataItem type="InterfaceBin">True </InputDataItem> <InputDataItem type="FunctionalBin">True</InputDataItem> //Only for ARIES data Source and HCSL Operation (Here 6881 is HCSL operation) <InputDataItem type="SubFlowStep">CLASSHOT</InputDataItem> </InputData> </CattsTcaServiceRequest> //Note, update with correct SubFlowStep value. </pre>	Set output column value = False if you dont need specific column(s) in output	

[\[Return to Top\]](#)

TDX-TO-CSV Utility

[\[Return to Top\]](#)

The **TDX-TO-CSV** utility converts all TDX files located under a share path that match a certain pattern to CSV file(s). Note that only conversion to CSV files (comma delimited files) is supported. The utility takes five arguments as illustrated in Figure TDX2.

1. **TDX Source Path**: The source path for the TDX files. All sub-folders of this path will be searched for matching TDX files

2. **Output Path:** The path where the CSV file(s) will be created. A "." denotes the current directory. Note that the name of the CSV files to create are specified in the TDX Conversion specification file.
3. **Config File Path:** The TDX to CSV Conversion Spec File. Several sample conversion spec files for known TDX files are provided in the drop-down help. Otherwise you may browse to the location of the specification file.

- *C502A691_1680* *C502A692_1680*

- A file containing a list of patterns with one pattern per line. E.g., **c:\file_patterns.txt**. For this case, you must also check option **Pattern field stores file containing Patterns** on the form. The contents of the file might be:

```
FILE_PATTERNS
*C502A691_1680*
*C502A691_1814*
```

- A pattern containing a date computation. E.g.,: ***1814*>=today-1***. Note that token **"*>=today-n*"** translates to all dates greater than today's date minus some number of days. If today is Feb 24th, 2015, then the following two TDX file patterns will be processed for token ***1814*>=today-1***:

```
*1814*20150224*
*1814*20150223*
```

To add the **TDX-TO-CSV** utility to your query, choose menu option **Query -> Add Table or Utility** from the Query design form. Then select Data Area -> **Utilities** on the **Add a View or Utility** window, and double-click the **TDX-TO-CSV** object. This is illustrated in Figure TDX1 below. You can then position the utility in your query using drag-and-drop. To configure the utility, double-click it. This invokes the TDX-TO-CSV utility form illustrated in Figure TDX2.

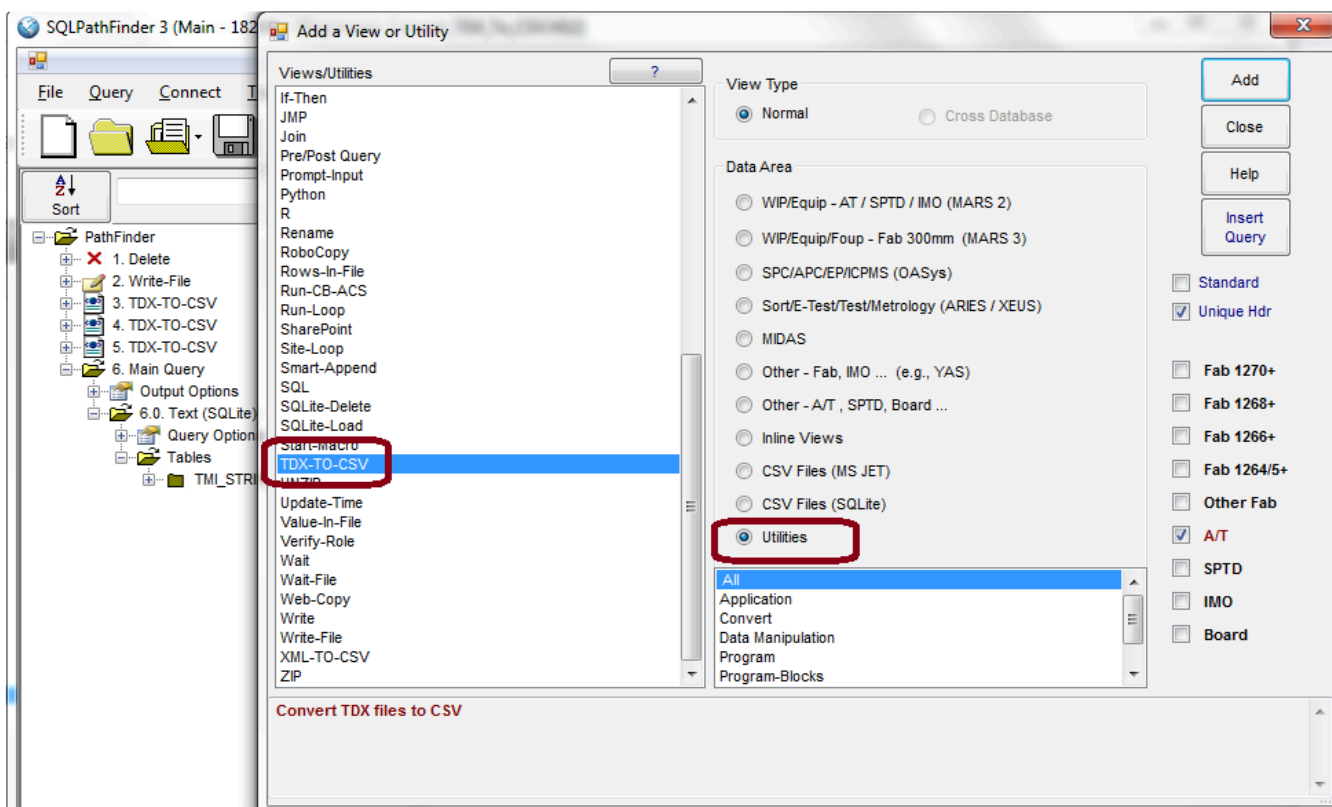


Figure TDX1 - Adding the **TDX-TO-CSV** Utility to your query.

Figure TDX2 - Configuring the **TDX-TO-CSV** Utility[\[Return to Top\]](#)

Option for Program-Block Utilities - Run Child Utils Even if Parent Utility is Disabled

[\[Return to Top\]](#)

As shown in Figure pb-1 below, there is a Query menu option labelled: **"Run Child utils even if parent util disabled"**. If this option is checked, then enabled child utilities (i.e., utilities inside program-block utilities), will be executed when the query is run, whether or not the parent utility is enabled or unassigned. If the option is unchecked, then child utilities will not be run if the parent utility is disabled or unassigned. For any existing query, this option will be set until a user turns it off thus mimicking current behavior. For any newly created query, this option will be disabled as it is more intuitive behavior from a visual standpoint. The user can always turn that option on.

E.g., in Figure pb-2, if Query option: **"Run Child utils even if parent util disabled"** is checked, then steps 4 and 5, if enabled, will be executed, even though the For-Loop in Step 2 is disabled.

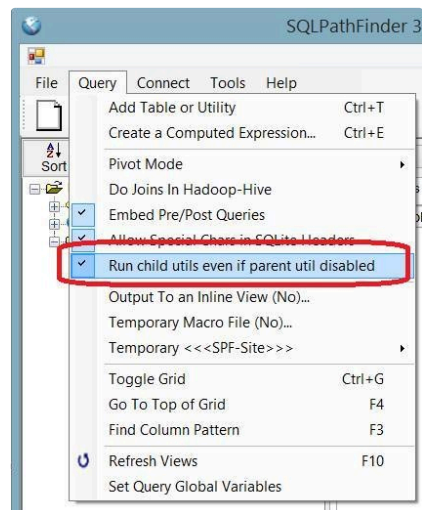
Figure pb-1: Query Option **Run Child utils even if parent util disabled**

Figure pb-2: Sample For-Loop

[\[Return to Top\]](#)

If-Then-Else Utility

[\[Return to Top\]](#)

associated with an If-Then-Else utility will be indented underneath the utility for ease of readability. This is illustrated in Figure if-1.

The If-Then-Else utility optionally supports two AND/OR conditions. This is illustrated in Figure if-2. In Figure 2, two conditions are connected using an AND clause, meaning both conditions must be TRUE. If an OR clause were used, then either condition must be true for the If-Then utility to translate to true. You will be able to use Environment Variables, Global Variables, command-line variables as well as constants in the variable portion of an IF-Then utility test. To use Global, Command-line or Macro variables, surround the variable with VAR(). This is illustrated in Figure if-2. In Figure if-2, command-line variable cl_user (i.e., VAR(<<<cl_user>>>)) is compared to value jmclarke. A command-line variable is a variable passed to the SQLPathFinder executable. Command line variables start with "cl_".

Note that if an IF/THEN condition cannot be evaluated due to a data type mismatch (e.g., comparing two strings using the EQ operator rather than an EQS operator), the SQLPathFinder extract engine will abort the job.

Figure if-1 - If-Then-Else Utility

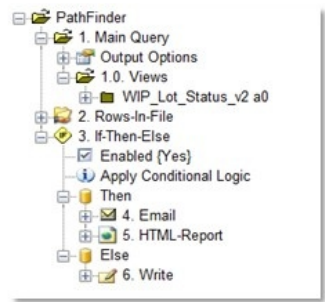
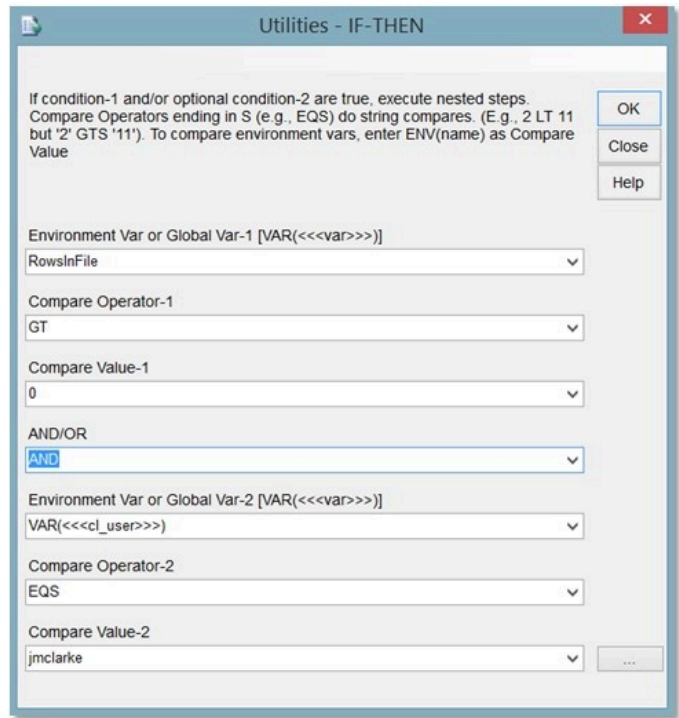


Figure if-2 - If-Then-Else Utility Configured with Two Conditions connected by an AND clause, and that leverage an Environment Variable (RowsInFile) and a Command-Line Variable (CL_USER)



[\[Return to Top\]](#)

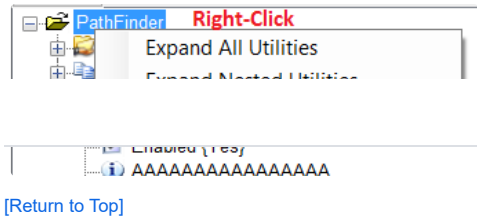
Expanding and Collapsing Utilities

[\[Return to Top\]](#)

To facilitate opening and collapsing utilities, especially when there is deep nesting, you may right-click over the "**PathFinder**" Root node on the Query Design form, and choose one of the Expand and collapse options as shown in Figure ec-1. Option:

- **Expand All Utilities** - expands all utilities fully allowing you to see a utility's label as well as its enable attribute
- **Expand Nested Utilities** - expands all utilities, regardless of nesting, without showing the utility label and enable attribute
- **Collapse All Utilities** - Collapses all utilities

Figure ec-1 - Expanding and Collapsing Utilities



Deleting, Moving, or Running a Block of Utilities

[\[Return to Top\]](#)

To facilitate job development using Utilities, SQLPathFinder now supports a **Begin-Block** utility that lets you cluster (or nest) a group of utilities or steps, and then move those steps, delete those steps or run those steps. Figure 1 shows how you can add a **Begin-Block** utility and its associated **End-Block** to your query.

You may then proceed to drag your Begin-Block and End-Block utilities so they bound a contiguous series of steps. Provided, the Begin-Block and End-Block utility have the same parent, you may then right-click the Begin-Block utility and choose option **Nest** from the drop-down menu. This nests all steps between the Begin-Block and End-Block utility underneath the Begin-Block step. This is illustrated in Figure 2a and 2b below.

Having nested, you may then right-click the Begin-Block utility and choose option **Remove** or **Run** to delete the block of utilities or to execute those steps respectively. You may also drag the Begin-Block to another area of your query in order to move that group of steps.

Figure 1 - Adding a Begin-Block Utility to your Query.

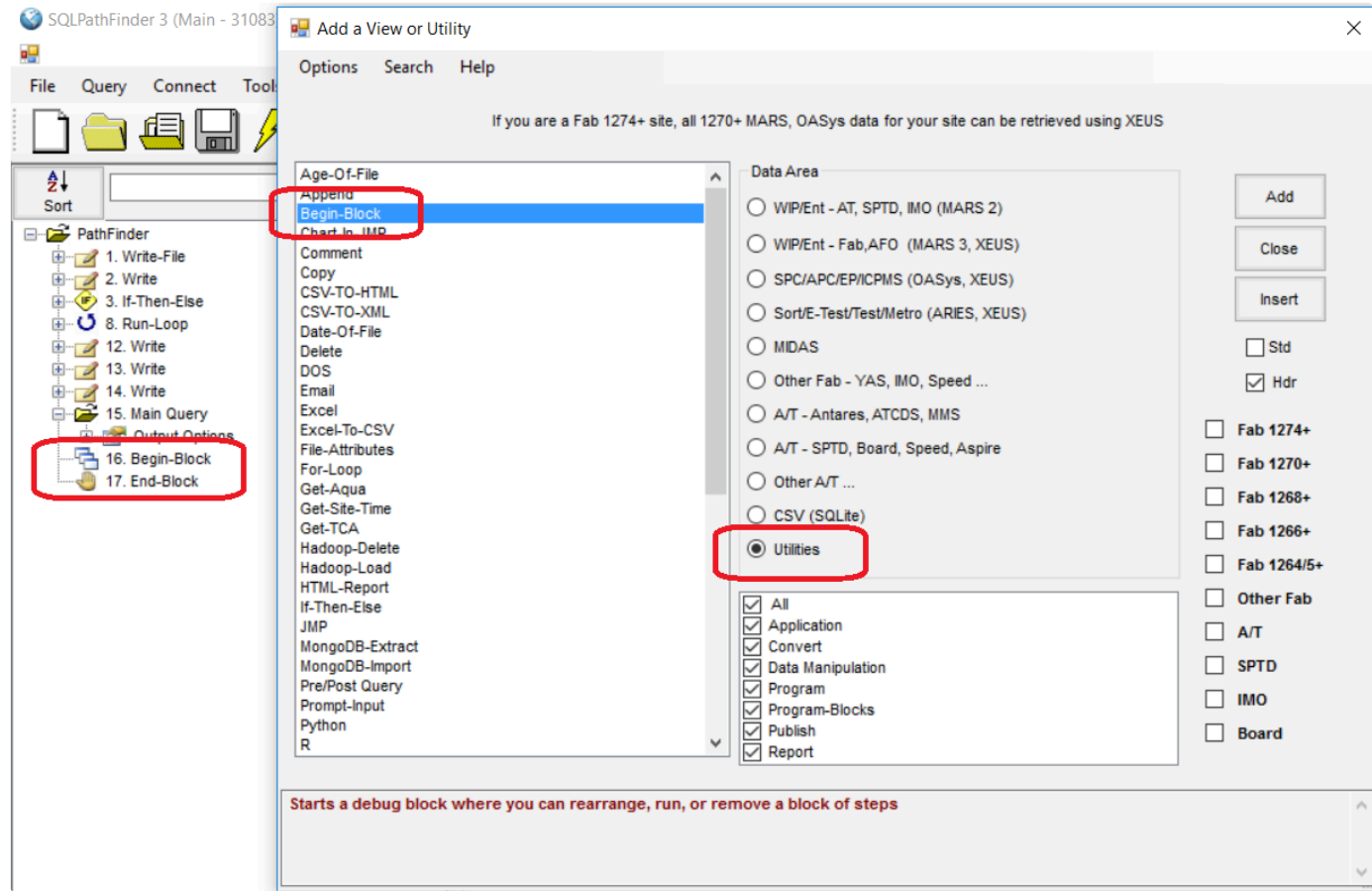


Figure 2a - Nesting Utilities underneath the Begin-Block Utility

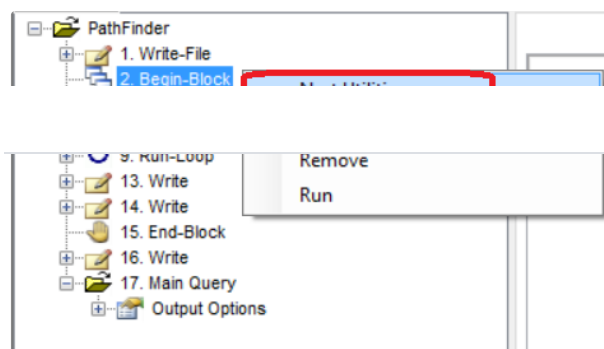
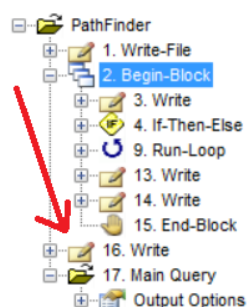


Figure 2b - Nesting Utilities underneath the Begin-Block Utility



Mongo-Import Utility

[\[Return to Top\]](#)

The Mongo Import utility imports a CSV or a JSON file into a MongoDB database. Rows can be inserted (Import Mode= Insert) or updated (Import Mode = Upsert) based on your identified key columns (opt. Upsert Field(s)), otherwise an _id column will be assumed to be the primary key. You can also insert/update fields into rows of an existing MongoDB database collection (i.e., Import Mode=Merge).

Field Options are:

Input Option	Description
Connection	This is the connection to the Mongo Database. This is an @ delimited string of format: server:port@database@@username@password. E.g., atdspwidelrds1:27017@ATM_KS@@un@pw
Input File	This is the path and name of the CSV/TAB or JSON file to import. The file extension determines the type of data to be imported. A comma delimited file should have an extension of .csv, a tab delimited file should have an extension of .tab, and a json file should have an extension of .json.
Collection	This is the name of the MongoDB Collection to insert data to.
Number of Workers	lets you specify multiple parallel threads for the import to speed data loading
Import Mode	Controls whether you wish to insert rows, update rows or merge rows. Valid values are: insert: Insert the documents in the import file. mongoimport will log an error if you attempt to import a document that contains a duplicate value for a field with a unique index, such as _id. upsert: Replace existing documents in the database with matching documents from the import file. mongoimport will insert all other documents. merge: Merge existing documents (i.e., insert or update fields in a matching document) that match a document in the import file with the new document.
opt. Upsert Fields	Modifies the import process to update existing objects in the database if they match based on the specified field list.
Other Arguments	For <i>Other arguments</i>, option --ignoreBlanks is common to not load a field if it stores an empty string. In addition, for CSV/TAB file imports, option --columnsHaveTypes lets you include field data types as part of field column headers. Otherwise Mongo will try and assume the datatype of the field based on actual data. An example of column headers with datatypes used with option --columnsHaveTypes are as follows: <pre>_id.string(),lot.string(),operation.string(),retest.int32(),tool.string(),timestamp.date_oracle(yyyy-mm-dd hh24:mi:ss),tool_group_code.string(),bga_no.int32(),visual_id.string()</pre> The import is based on the mongoimport utility. Hence, you can get more information on this at: https://docs.mongodb.com/manual/reference/program/mongoimport/

File-Compare Utility

[\[Return to Top\]](#)

The File-Compare utility lets you compare two files and will report any differences. To add this utility to your query, choose menu option **Query -> Add Table or Utility** on the Query design form, select **Data Area Utilities** on the **Add a Table** window, and double-click the **File-Compare** item to add it to your query. You can then position the utility using drag-and-drop.

The File-Compare utility lets you optionally do case insensitive comparisons between two files, and can mask out dates of specific formats. As of the initial release, date masking can be optionally done for dates of format: YYYY-MM-DD HH:MI:SS. The utility also optionally supports ending after some number of file differences are encountered, and can optionally email the difference log to the account that ran the job. Lastly, the utility can be configured to error if any differences are found.

Utilities - FILE-COMPARE

Compares two files. Optionally specify a case insensitive search, a date format to mask date differences, whether to notify you if there are differences, and a maximum difference count

☒ Continue on Error

OK
Close
Help

File to compare

2.txt

...

File to Compare

3.txt

...

Case Insensitive Search?

Y

Max Differences

Date Mask (opt)

YYYY-MM-DD HH:MI:SS

Email if Mismatch?

Y

IDEAL-Table-Config Utility

[\[Return to Top\]](#)

The IDEAL-Table-Config utility lets one configure an IDEAL table based on a configuration file. The utility requires the user to have IDEAL installed on their computer. One benefit of this utility is that variables can be configured based on column patterns instead of individual variable names. The output of the Ideal-Table-Config utility is an IDEAL document (.ide).

Figure 1 - IDEAL-Table Config-Utility Form

Utilities - IDEAL-TABLE-CONFIG

Create an Ideal Load File Specification based on a CSV/TAB file to load and a Variable Configuration file

OK
Close
Help

CSV File to Load to Ideal

...

Variable Configuration File

...

Ideal Output Specification File

ideal_out.ide

...

The utility requires paths to 3 files.

- 1. CSV File to Load to Ideal: This is the file containing data to load to IDEAL. This file is referenced by the IDEAL document so it should be in a fixed location. If an ".ide/.idt" file is opened in Ideal and the raw data was moved, the user will be prompted to locate it.
- 2. IDEAL Output Specification File: This is the output IDEAL document with the variables configured. File can be a .ide (Ideal document) or .idt (Ideal document text) file.
- 3. Variable Configuration File: This is a user supplied configuration file that will define the type of configuration each variable will have in IDEAL.

Figure 2 - IDEAL Variable Configuration Table

commonality_data.csv - Variables						
						
1		lot	String	Categorical	☐ OneVsTheRest	23849 Input
2		operation	Number	Numeric	☐ Interval	23849 Ignored

There are two columns in IDEAL that cannot be configured: "Data type" and "NonMissing". These columns show the user what data IDEAL found during the data table inspection. Since these columns can't be modified they are not expected in the variable configuration file.

A sample variable configuration file can be found here: [Ideal config table example](#).

Figure 3 - Sample Variable Configuration File.

name	variable_type	split_type	role
sample_variable_1	Numeric	1	Output
sample_variable_2	Categorical	0	Ignored
sample_variable_3	Ordinal	1	Input

The Variable Configuration File needs to contain at least a "name" column and one of the other configurable columns. The check boxes in split type can be configured with a "1" for "Checked" and "0" for "Unchecked". The words "Checked"/"Unchecked" also work.

If any variable in the configuration table is misspelled or doesn't exist in the data file, it will just be ignored. In the same way if the user tries to set a configuration that is not allowed in IDEAL, for example "Variable type" as "Groups", the change will be ignored. The variable configuration file does not require the columns to be in order and will ignore other columns not needed to configure the table.

One of the main features of this configuration utility is the ability to configure variables based on patterns rather than exact names. This is achieved using Regular expressions. There are multiple tutorials and [cheat sheets](#) available online which can be referenced for advanced usage. Four use cases are described below:

- **Contains:** Any text in the regular expressions is treated as "Contains". This means that if "lot" is placed in the "name" column, all columns containing "lot", such as "lot_name" or "lot_number", will also be configured in the same way.
- **Begins with:** Adding "^" to the beginning of a text will match it only when it starts with that text. For example "^abc" will find "abc", "abcd", "abc123", etc.
- **Ends with:** Adding "\$" to the end of a text will match it only when it ends with that text. For example "abc\$" will find "abc", "endsinabc", "123abc", etc.
- **Exact Match:** Using the begins and ends examples above, you can surround a name with "^" and "\$" and it will be configured only when an exact match is found. E.g., "^lot\$" will only find the exact text "lot". Field "lot_name" would not be configured.

WRITE-File and WRITE-File-Immediate Utilities

[\[Return to Top\]](#)

The **Write-File** utility lets you write text to a file when the query executes. It could be used to create, for example, a Python script that you wish executed later in the job using the Python utility. The **Write-File-Immediate** utility also writes text to a file, but occurs before the query is "compiled and executed. The latter is useful, for example, when used in conjunction with the **prompt-Input** utility. Similar to **Write-File-Immediate**, the **Prompt-Input** utility executes before the query is "compiled" and run. The **prompt-Input** utility requires an input specification file to define the input prompts. If you wish to create that input specification file in the same query as the **prompt-input** utility, use the **Write-File-Immediate** utility to generate the specification file. This ensures the specification file is created before it is needed by the **prompt-input** utility.



Note that since the **Write-File-Immediate** utility occurs before run-time, it will not translate any Macro Variables. It does support Global Variables, however. The **Write-File** utility is executed at run-time and does support Global as well as Macro variables.

Screen-and-Visualize-Features Utility

[\[Return to Top\]](#)

[Link to training provided by SK Chew during ProCA training Oct-2020](#) → [Video_Link](#)

[Link to Video Demonstration on Screen-and-Visualize-Features](#) → [Video_Link](#)

[Link to Presentation on Screen-and-Visualize-Features](#) → [PowerPoint](#)

[Link to CATTs POR Variable Config File](#) → [Link](#)

[Link to optional config.ini file \(needed only to modify html default look\)](#) → [Link](#)

This utility speeds the screening of features (variables) from a commonality data set in order to quickly determine if the main features need further analysis or if they can be discarded as not relevant. The output of the utility is an HTML report that shows thumbnails of the Correlation Plot (described below). Clicking the thumbnail takes the user to another page with a full size version of the plot and any other charts that can be generated based on the feature data type and the data set used.

The utility uses the power of [IDEAL Machine Learning](#) to rank feature importance and then creates the visualization report. Version **3.20.1** or later is required when using parquet data sets since previous versions would not recognize dates properly.

[Screen-and-Visualize-Features Utility Form](#)

Figure 1 -Screen-and_Visualize-Features-Utility Form

"Options -> Minimum Var. importance %" defines the smallest variable importance percentage to accept (1 to 100=highest). "Chart Config File" is a file with chart attributes and "Save Important Vars file" creates a smaller input file storing important vars only for post analysis

CSV File to Load to Ideal
raw_data.csv

Variable Configuration File
variable_config.csv

Ideal Output Specification File
ideal_out.ide

HTML Output File
SPFCommonality.htm

Data Type (e.g., CATTs)
CATTs

Chart Config File (Opt)
myconfig.ini

Save Important Vars File (Opt)
imp_cols.csv

OK
Close
Help

Figure 1 shows the utility form.

-The first argument is the data csv to load. Data can be a csv or tab separated file.

-The second argument is the variable configuration file. A complete explanation of how the Variable Configuration File works can be found in the [IDEAL-Table-Config Utility](#). A standard variable configuration file for CATTs data based on SK Chew ProCA training can be found here → [Link](#)

-The third argument is the IDE file to be produced. This will be the ideal document with the variables configured as per the Variable Configuration file. This IDEAL file is produced in case further analysis is needed in the IDEAL application. . File can be a .ide (Ideal document) or .idt (Ideal document text) file.

-The fourth argument is the path and name to the report output html to be produced.

-The fifth argument is the data source. The options are KitchenSink, CATTs or Other. The data source will be used to find other columns that are relevant to the feature being graphed and produce additional graphs when possible. If Other is selected only the Correlation Plot will be produced for each feature. The general charts section won't be produced either (see report section below).

-The sixth argument is optional, it allows the user to input a configuration file that controls the report and graphs. There is a section below that explains the configuration file and its common options. It also contains links to a config file with all possible options.

-The seventh and last argument is also optional. The argument provides an option to specify a file where a smaller version of the data set provided will be saved. This smaller version will only contain the features found important by IDEAL and its related features in the data set. For example if entity at 1210 is found important, the out date and carrier locations for this operation will also be saved (in case they're found in the data set). This allows the user to have a reduced size file with which he can use for any further analysis in JMP or any other program. The more features in the data set the bigger the difference that can be achieved between this file and the original one.

-There is an option in the top menu that sets the variable importance cutoff for displaying charts in the report. The default is 5% but can be decreased to show more graphs, or increased to show less.

Figure 2 - Variable Importance Algorithms

Utilities - Screen and Visualize Features

Options

creates a smaller input file storing important vars only for post analysis

Ideal_Forest
Feature_Selection
Decision_Tree_1_Surrogate

Close
Help

CSV File to Load to Ideal
test_data.csv

Variable Configuration File
ideal_config.csv

Ideal Output Specification File
ideal_out.ide

HTML Output File
SPFCommonality.htm

Data Type (e.g., CATTS)
CATTS

Chart Config File (Opt)
None

Save Important Vars File (Opt)
None

Figure 2 shows the last option in the form, it allows you to choose from the two algorithms currently implemented from Ideal to obtain the variable importance. The default option is Ideal Forest but Feature Screening and Decision Tree with 1 surrogate are also available.

Algorithm Parameters

Ideal_Forest: Uses 200 iterations and out of bag error estimation with 60% sub-sampling

Feature_Selection: Uses 20 series with 20 trees, 1000 trees for residuals, 6 iterations and 60% Random sub-sampling

Decision_Tree_1_Surrogate: Single tree, 1 surrogate and cross validation

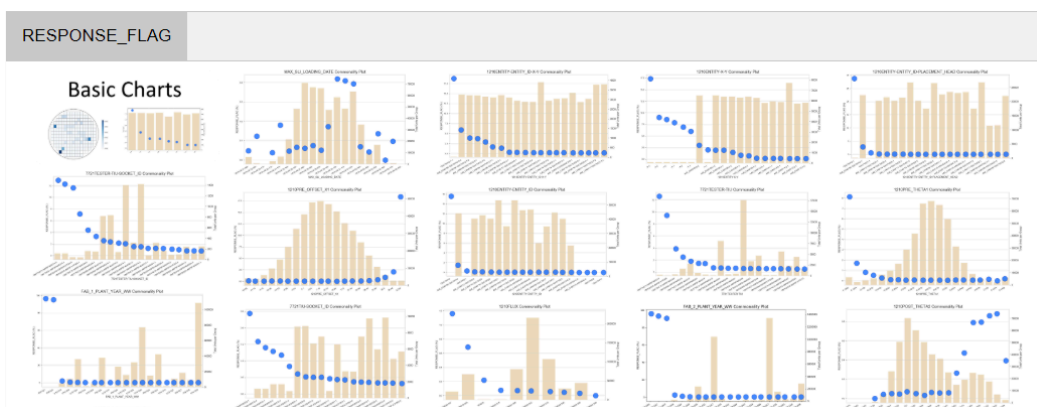
Feature Screening Report

The feature screening report is an HTML page that will open once the utility finishes. A sample report can be seen below.

Figure 3 -Feature Screening Report

Feature Screening Report

Created: 2019-09-05 08:54:00



The report contains:

Title and subtitle - this can be changed in the config.ini

Tabs for each output variable - In this case RESPONSE_FLAG is the only output, multiple output variables can be marked in the variable configuration file

Basic Charts - This is a section where general charts can be generated regardless of the variable importance can be found, for example, GET commonality of Wafer/Strip maps. This is only available from CATTs or KitchenSink.

Correlation Plot

The main chart produced for all variables is a correlation plot. The plot can be produced for continuous or discrete variables. The blue dots marks the mean of each group. The scale can be read to in the left Y axis (Y1). The bars represent the total units in the group, the scale can be read on the right Y axis (Y2).

Ack. SK Chew for providing the data for the correlation plots based on his ProCA Class - Course Codes: 10000493 and 10000495

Figure 4 - Correlation Plot for a continuous variable.

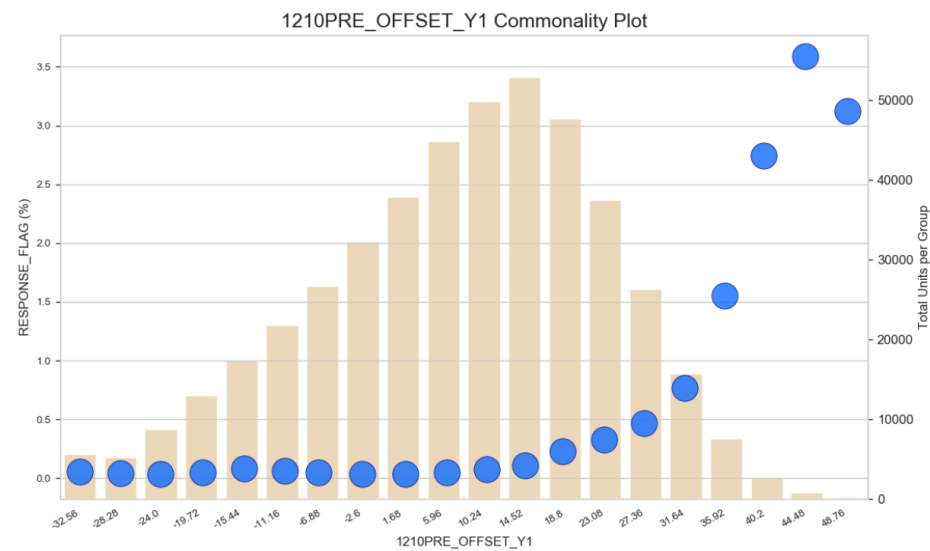


Figure 3 shows a correlation plot for a continuous variable. In this case the variable is split into x number of groups (20 is the default), the center of each group is displayed in the x-axis. The mean and volume in each group is calculated and graphed.

Figure 5 - Correlation Plot for a discrete variable.

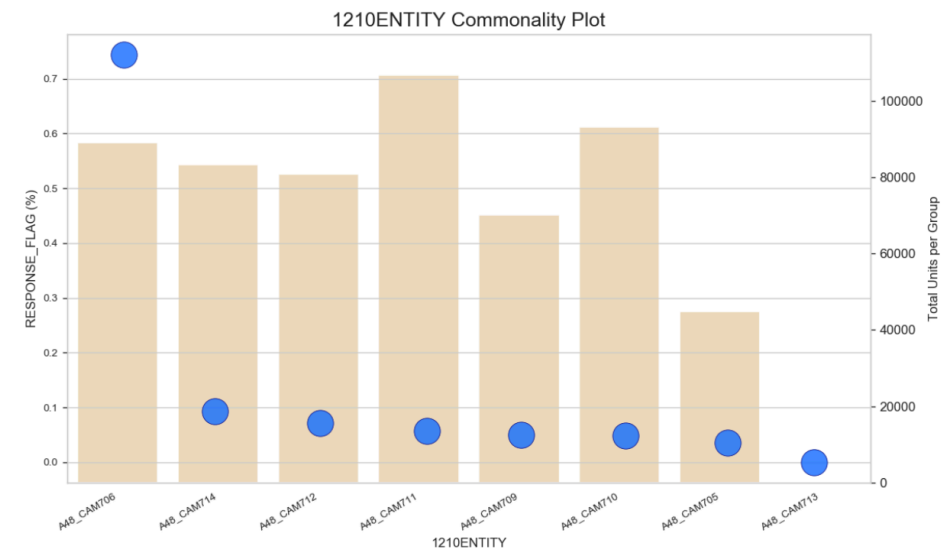


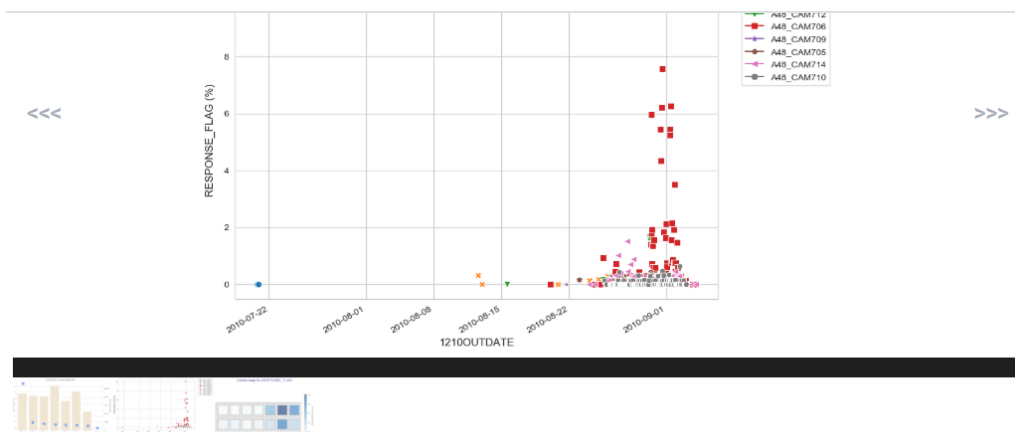
Figure 4 shows a correlation plot for a discrete feature. The different values in this discrete feature will be the graphed. The mean response of each groups will be sorted ascending or descending based on the config.ini file (default is descending) and only the top x groups are shown (default is 20 groups). In most cases there will be one or two groups that cause the commonality. 20 are show to be able to compare the normal and bad groups.

Feature Page

When any thumbnail in the main page is clicked the feature page will be open. The page is divided into two areas, the top displays the active graph and the bottom has all the chart that were possible to be produced based on the type of variable and the data available in the data set. Clicking any of the thumbnails in the bottom section will make that graph active and a large version is shown in the top view.

Figure 6 - Feature Page.

Charts for: 1210ENTITY

Config.ini file

This is the config file that allows the user to configure the look of the report and charts. The configuration file can be as simple as a single header with a single value to be changed or as complex as the sample config.ini file in the link below. Even if the sample config.ini file contains all options, the best practice is to **leave only the options that need to be modified** and delete any other option that you want to leave as default.

The config.ini file has multiple sections, Report Configuration, Correlation plot configuration, General Chart Configuration, Carrier Map Configuration and Wafer Map Configuration.

Below some of the options are described, the most self intuitive ones are not explained.

A config.ini file with most options available can be found here → [Link](#).

Report Configuration:

ADD_GENERAL_CHARTS - This option allows you to add or remove the general charts group that shows in the first thumbnail in the report

COLUMNS - This option enables you to configure the amount of thumbnail columns that want to be displayed in the main report

REPORT_TITLE - Main report title

REPORT_SUBTITLE - Report subtitle, the word "default" will write the date the report was created. Any other value will write that value to the report subtitle.

Correlation Plot Configuration:

MAX_BINS - This option configures the maximum groups that will be displayed in the correlation plot.

MIN_BIN_DATA - This is the minimum data in a group that will make the mean dot show.

SORTTYPE - Sorting order for x-axis groups.

RESP_IN_PCT - This makes the y-scale show in %, this is useful when the response variable is a 1 for rejects and 0 for good units. The mean for the response is the loss and this option would make it show as % loss.

CIRCLEFILLCOLOR - All colors in the config.ini file are in RGBA (red, green, blue, alpha) values need to be a tuple of float values in [0, 1], e.g., (0.12, 0.45, 1, 1)

XLABELROTATION - Rotation angle for the x-axis

General Chart Configuration:

FIGSIZE - Figure size for charts like boxplot or violin plots

TIMEFIGSIZE - Figure Size for Time Charts

JITTER - Jitters dots in boxplots

MIN_BIN_DATA - This is the minimum data in a lot/group that will make the mean dot show in the time trend

SHOWLINE - Changing this option to True would create a line in the time charts connecting dots

Carrier Map Configuration:

ISSTRIP - This tells the graph to remove the spacing between units and create a gap in the middle to show the clusters

BORDER_PROP - proportion of the graphing area that is dedicated to the border

UNITSPACINGX - proportion of the axis dedicated to space between units

STRIPGAP_PROP - proportion of the axis dedicated to space between strip clusters

COLOR_MAP - color map to be used for the carrier response values

Wafer Map Configuration:

DIAMETER - Wafer diameter

MINLOCX - Minimum Wafer location in X

MAXLOCX - Maximum Wafer location in X

MINLOCY - Minimum Wafer location in Y

MAXLOCY - Maximum Wafer location in Y

DIE_PITCH_X - Die X pitch

DIE_PITCH_Y - Die Y pitch

COLOR_MAP - color map to be used for the wafer map

No labels

